LAPORAN PEMROGRAMAN BERORIENTASI OBJEK PADA GEOMETRI ELIPS

Dosen Pengampu : Dr. Novrido Charibaldi S.Kom., M.Kom.

Urutan ke-10



Disusun Oleh :

Aziz Nabil Putra Darmawan	123240239
Ikhsan Wahyu Endriarto	123240241
Lahar Nuansa Putra	123240053
Muhammad Garnida Barreta N	123240230
Muhammad Hasan Rabbani	123240005

PROGRAM STUDI INFORMATIKA

JURUSAN INFORMATIKA

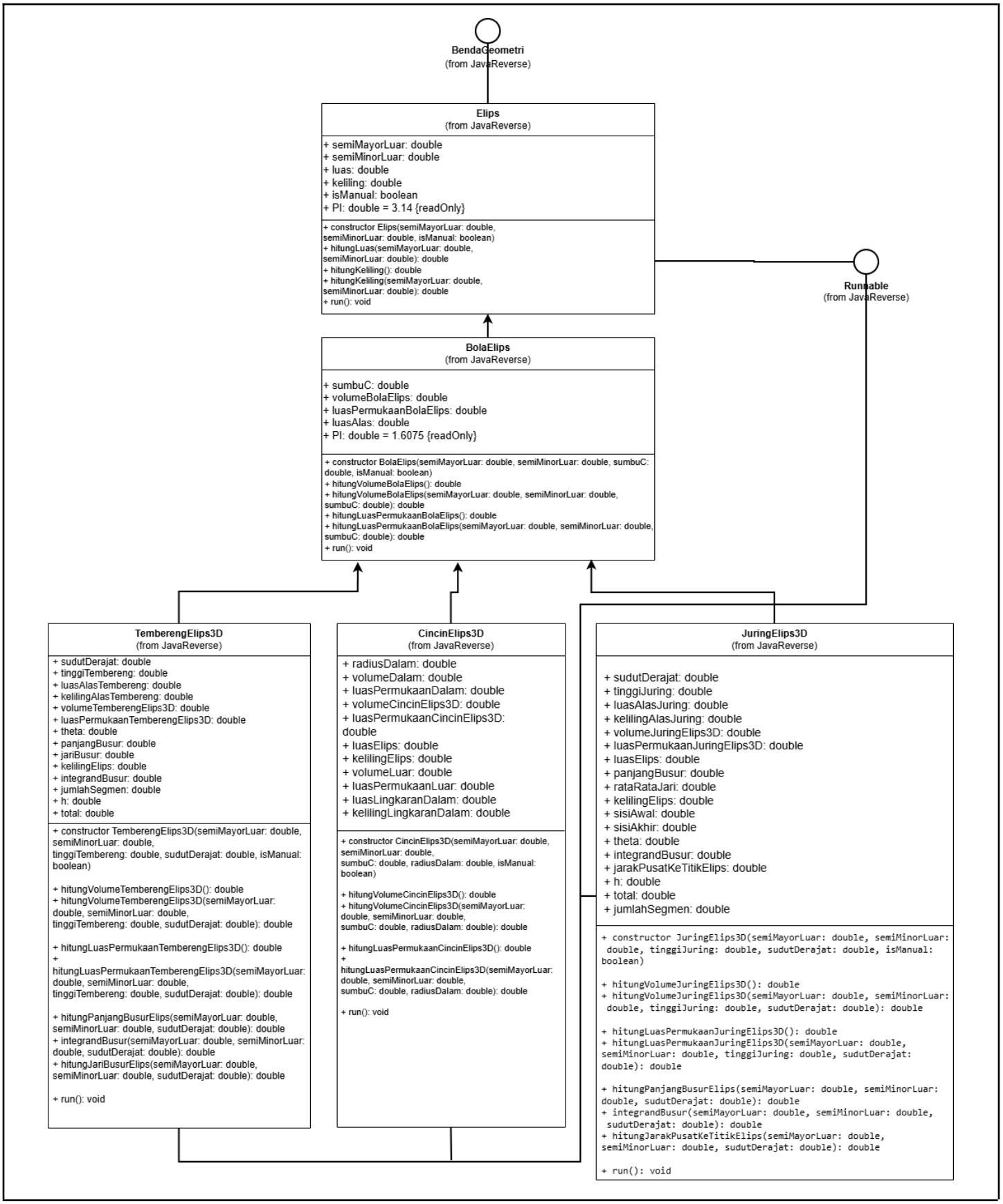
FAKULTAS TEKNIK INDUSTRI

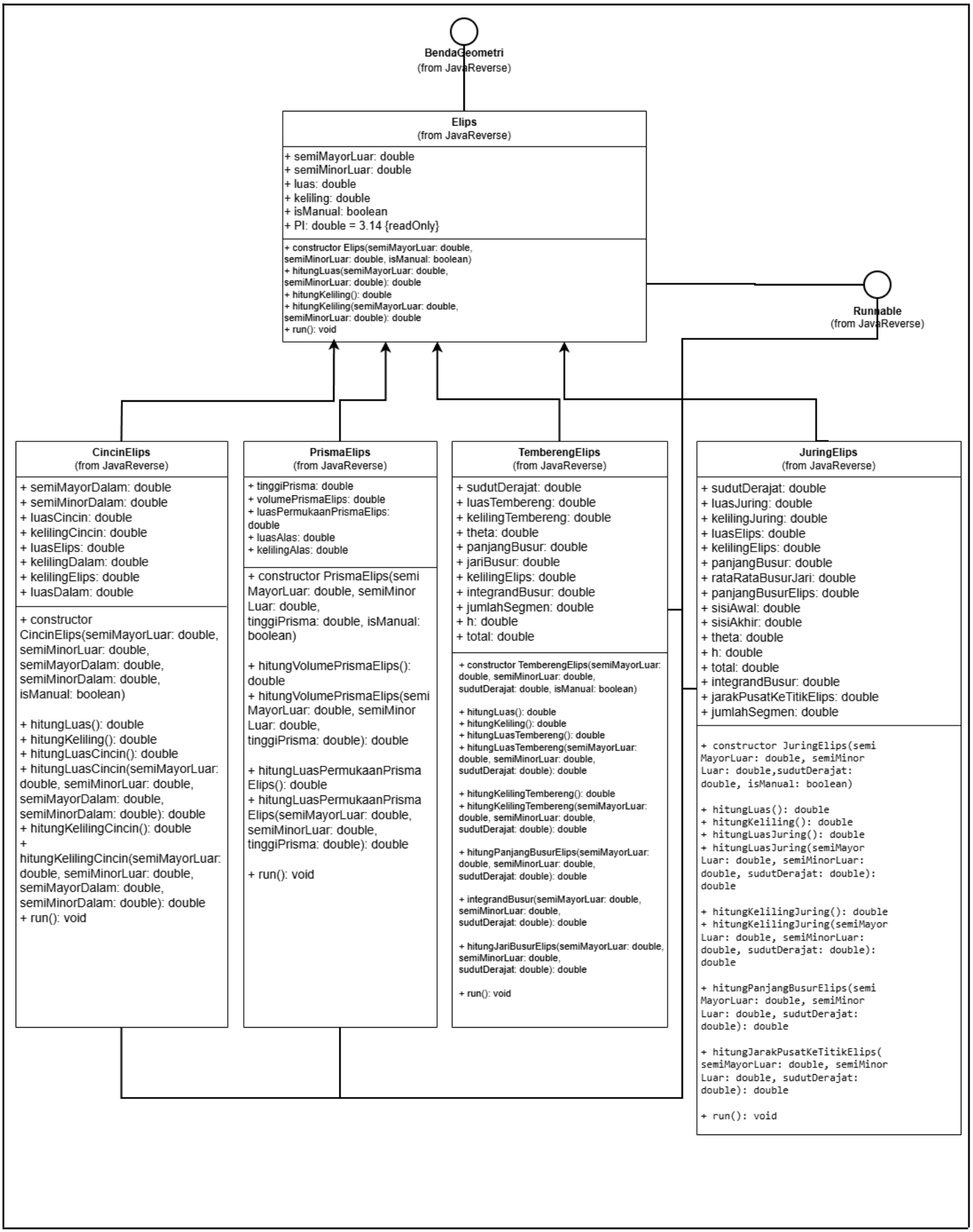
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN" YOGYAKARTA

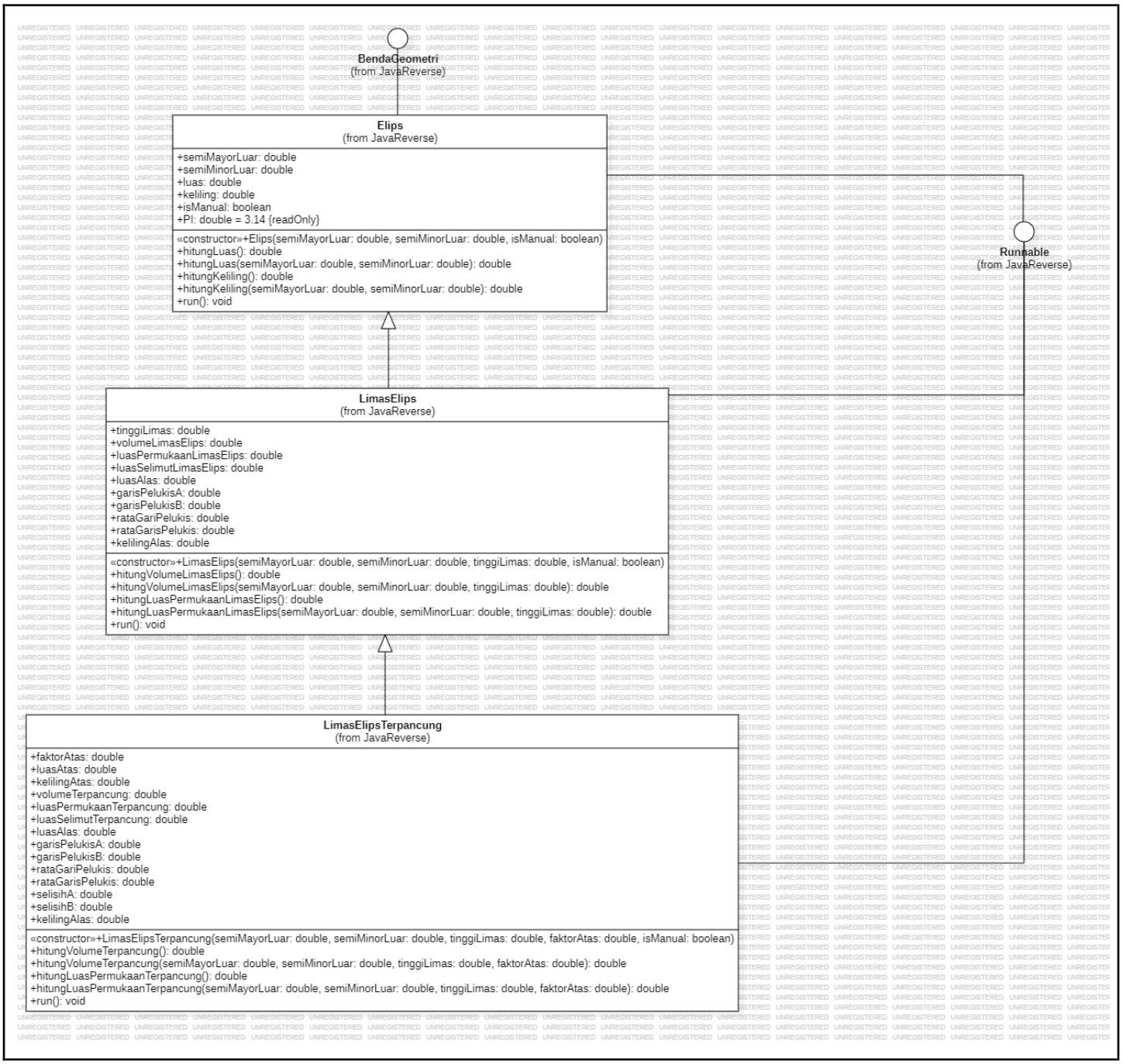
2026

BAB I

Unified Modeling Language (UML)

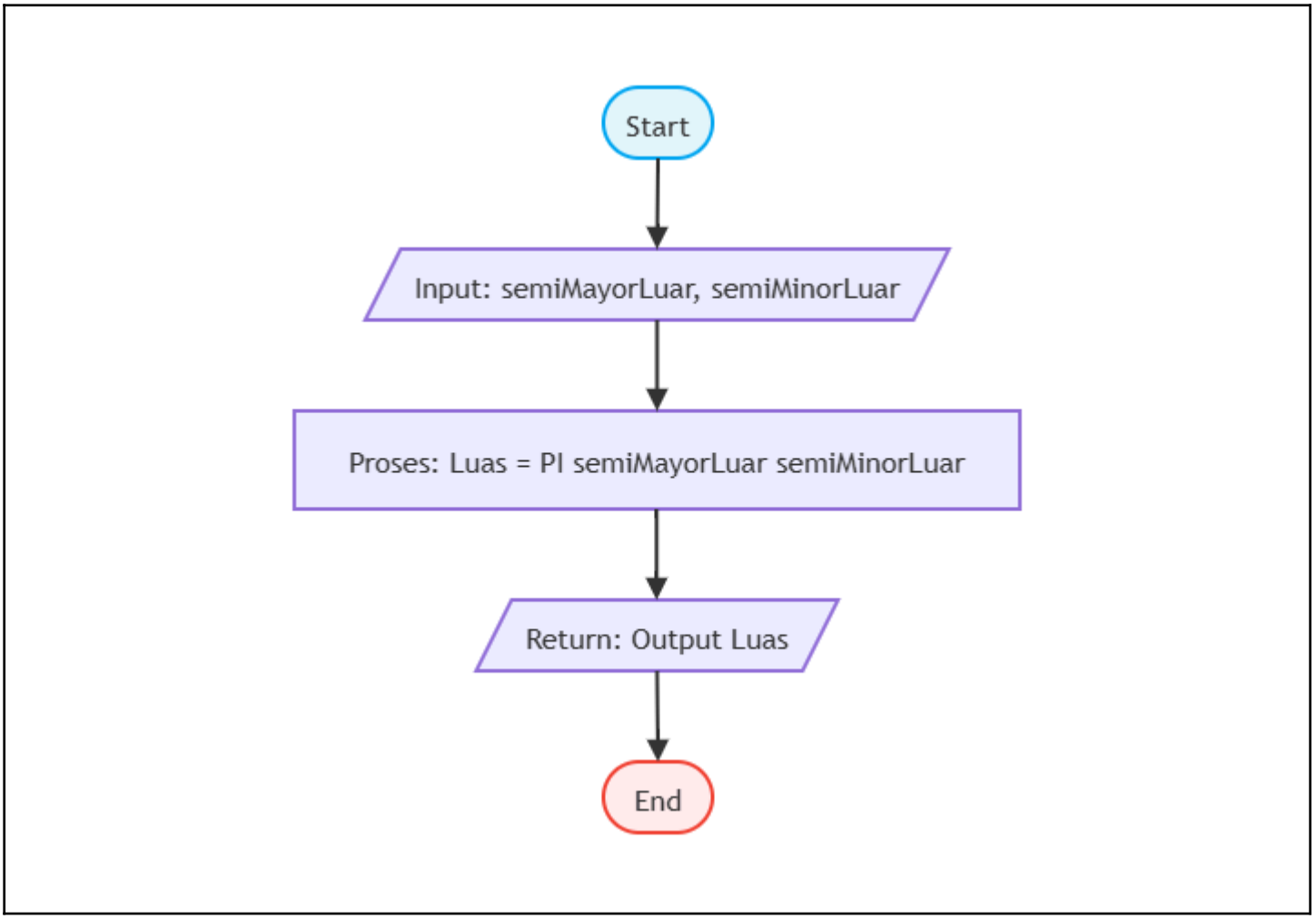



BendaGeometri
(from JavaReverse)
Elips
(from JavaReverse)
+ semiMayorLuar: double
+ semiMinorLuar: double
+ luas: double
+ keliling: double
+ isManual: boolean
+ PI: double = 3.14 {readOnly}
+ constructor Elips(semiMayorLuar: double, semiMinorLuar: double, isManual: boolean)
+ hitungLuas(semiMayorLuar: double, semiMinorLuar: double): double
+ hitungKeliling(): double
+ hitungKeliling(semiMayorLuar: double, semiMinorLuar: double): double
+ run(): void
Runnable
(from JavaReverse)
CincinElips
(from JavaReverse)
+ semiMayorDalam: double
+ semiMinorDalam: double
+ luasCincin: double
+ kelilingCincin: double
+ luasElips: double
+ kelilingDalam: double
+ kelilingElips: double
+ luasDalam: double
+ constructor CincinElips(semiMayorLuar: double, semiMinorLuar: double, semiMayorDalam: double, semiMinorDalam: double, isManual: boolean)
+ hitungLuas(): double
+ hitungKeliling(): double
+ hitungLuasCincin(): double
+ hitungLuasCincin(semiMayorLuar: double, semiMinorLuar: double, semiMayorDalam: double, semiMinorDalam: double): double
+ hitungKelilingCincin(): double
+ hitungKelilingCincin(semiMayorLuar: double, semiMinorLuar: double, semiMayorDalam: double, semiMinorDalam: double): double
+ run(): void
PrismaElips
(from JavaReverse)
+ tinggiPrisma: double
+ volumePrismaElips: double
+ luasPermukaanPrismaElips: double
+ luasAlas: double
+ kelilingAlas: double
+ constructor PrismaElips(semiMayorLuar: double, semiMinorLuar: double, tinggiPrisma: double, isManual: boolean)
+ hitungVolumePrismaElips(): double
+ hitungVolumePrismaElips(semiMayorLuar: double, semiMinorLuar: double, tinggiPrisma: double): double
+ hitungLuasPermukaanPrismaElips(): double
+ hitungLuasPermukaanPrismaElips(semiMayorLuar: double, semiMinorLuar: double, tinggiPrisma: double): double
+ run(): void
TemberengElips
(from JavaReverse)
+ sudutDerajat: double
+ luasTembereng: double
+ kelilingTembereng: double
+ theta: double
+ panjangBusur: double
+ jariBusur: double
+ kelilingElips: double
+ integrandBusur: double
+ jumlahSegmen: double
+ h: double
+ total: double
+ constructor TemberengElips(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double, isManual: boolean)
+ hitungLuas(): double
+ hitungKeliling(): double
+ hitungLuasTembereng(): double
+ hitungLuasTembereng(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double): double
+ hitungKelilingTembereng(): double
+ hitungKelilingTembereng(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double): double
+ hitungPanjangBusurElips(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double): double
+ integrandBusur(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double): double
+ hitungJariBusurElips(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double): double
+ run(): void
JuringElips
(from JavaReverse)
+ sudutDerajat: double
+ luasJuring: double
+ kelilingJuring: double
+ luasElips: double
+ kelilingElips: double
+ panjangBusur: double
+ rataRataBusurJari: double
+ panjangBusurElips: double
+ sisiAwal: double
+ sisiAkhir: double
+ theta: double
+ h: double
+ total: double
+ integrandBusur: double
+ jarakPusatKeTitikElips: double
+ jumlahSegmen: double
+ constructor JuringElips(semiMayorLuar: double, semiMinorLuar: double,sudutDerajat: double, isManual: boolean)
+ hitungLuas(): double
+ hitungKeliling(): double
+ hitungLuasJuring(): double
+ hitungLuasJuring(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double): double
+ hitungKelilingJuring(): double
+ hitungKelilingJuring(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double): double
+ hitungPanjangBusurElips(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double): double
+ hitungJarakPusatKeTitikElips(semiMayorLuar: double, semiMinorLuar: double, sudutDerajat: double): double
+ run(): void

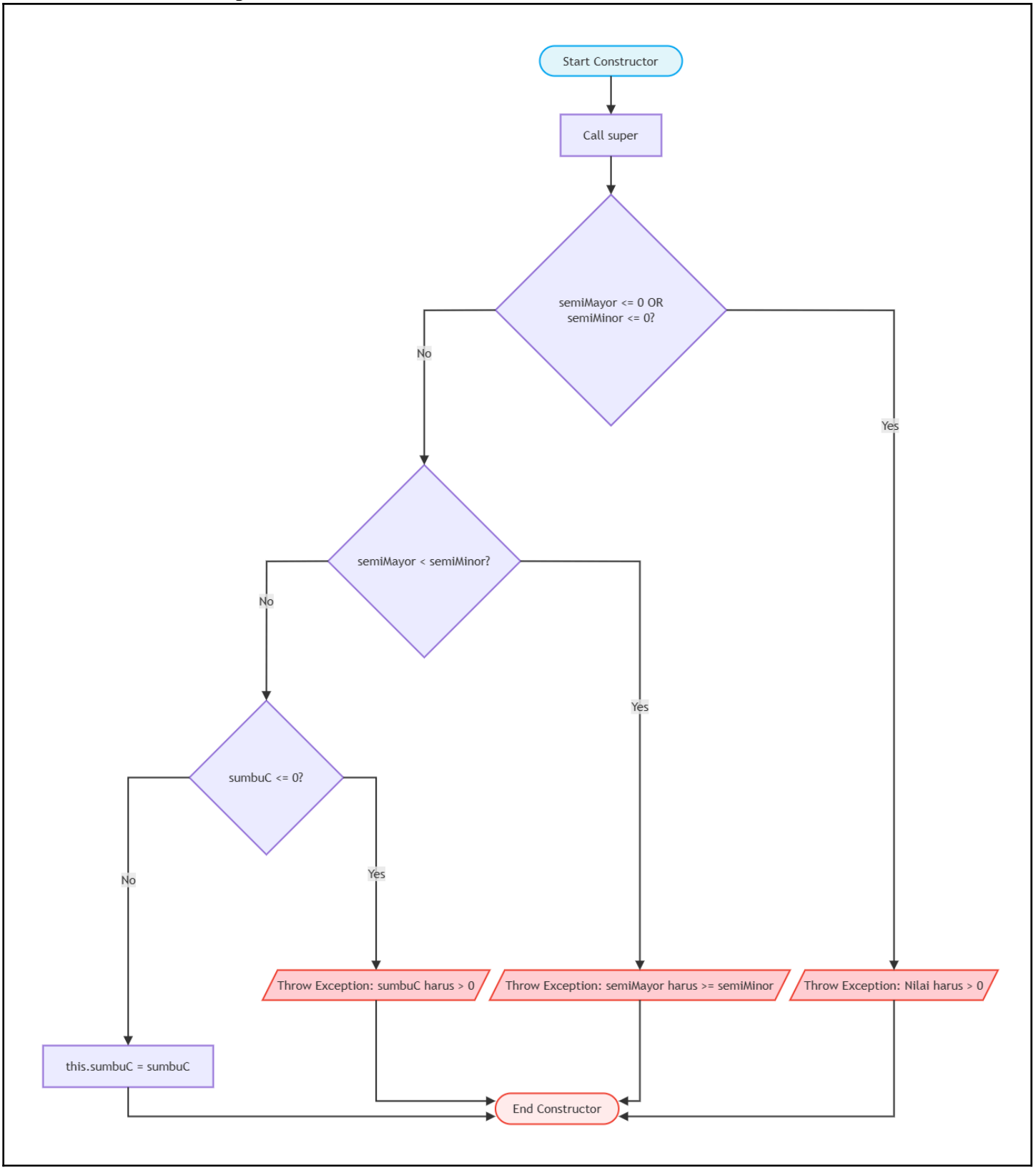

BendaGeometri
(from JavaReverse)
Elips
(from JavaReverse)
+semiMayorLuar: double
+semiMinorLuar: double
+luas: double
+keliling: double
+isManual: boolean
+PI: double = 3.14 {readOnly}
«constructor»+Elips(semiMayorLuar: double, semiMinorLuar: double, isManual: boolean)
+hitungLuas(): double
+hitungLuas(semiMayorLuar: double, semiMinorLuar: double): double
+hitungKeliling(): double
+hitungKeliling(semiMayorLuar: double, semiMinorLuar: double): double
+run(): void
Runnable
(from JavaReverse)
LimasElips
(from JavaReverse)
+tinggiLimas: double
+volumeLimasElips: double
+luasPermukaanLimasElips: double
+luasSelimutLimasElips: double
+luasAlas: double
+garisPelukisA: double
+garisPelukisB: double
+rataGariPelukis: double
+rataGarisPelukis: double
+kelilingAlas: double
«constructor»+LimasElips(semiMayorLuar: double, semiMinorLuar: double, tinggiLimas: double, isManual: boolean)
+hitungVolumeLimasElips(): double
+hitungVolumeLimasElips(semiMayorLuar: double, semiMinorLuar: double, tinggiLimas: double): double
+hitungLuasPermukaanLimasElips(): double
+hitungLuasPermukaanLimasElips(semiMayorLuar: double, semiMinorLuar: double, tinggiLimas: double): double
+run(): void
LimasElipsTerpancung
(from JavaReverse)
+faktorAtas: double
+luasAtas: double
+kelilingAtas: double
+volumeTerpancung: double
+luasPermukaanTerpancung: double
+luasSelimutTerpancung: double
+luasAlas: double
+garisPelukisA: double
+garisPelukisB: double
+rataGariPelukis: double
+rataGarisPelukis: double
+selisihA: double
+selisihB: double
+kelilingAlas: double
«constructor»+LimasElipsTerpancung(semiMayorLuar: double, semiMinorLuar: double, tinggiLimas: double, faktorAtas: double, isManual: boolean)
+hitungVolumeTerpancung(): double
+hitungVolumeTerpancung(semiMayorLuar: double, semiMinorLuar: double, tinggiLimas: double, faktorAtas: double): double
+hitungLuasPermukaanTerpancung(): double
+hitungLuasPermukaanTerpancung(semiMayorLuar: double, semiMinorLuar: double, tinggiLimas: double, faktorAtas: double): double
+run(): void

BAB II
Perancangan Alur Sistem (Flowchart)

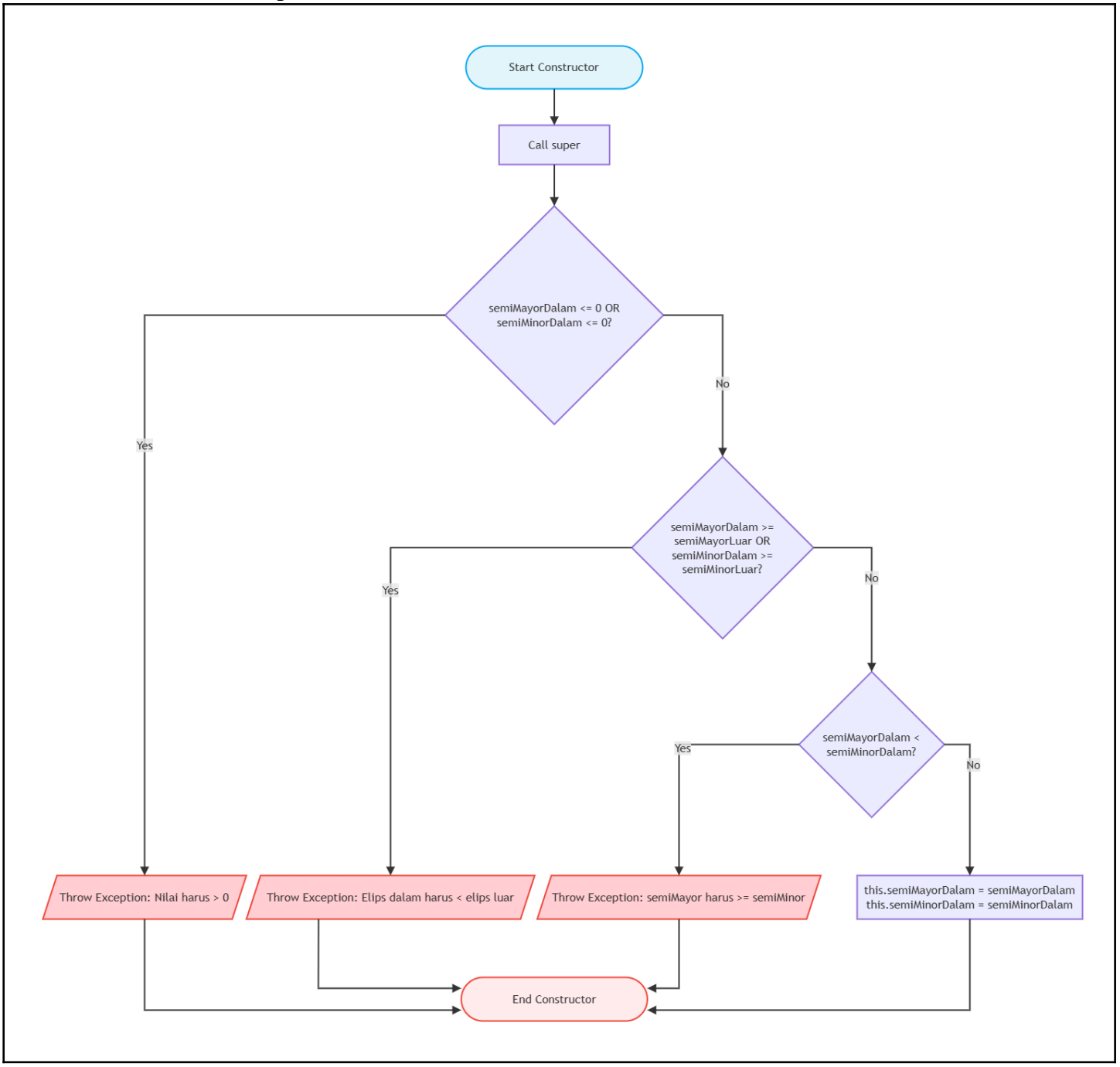
2.1 Flowchart BendaGeometri



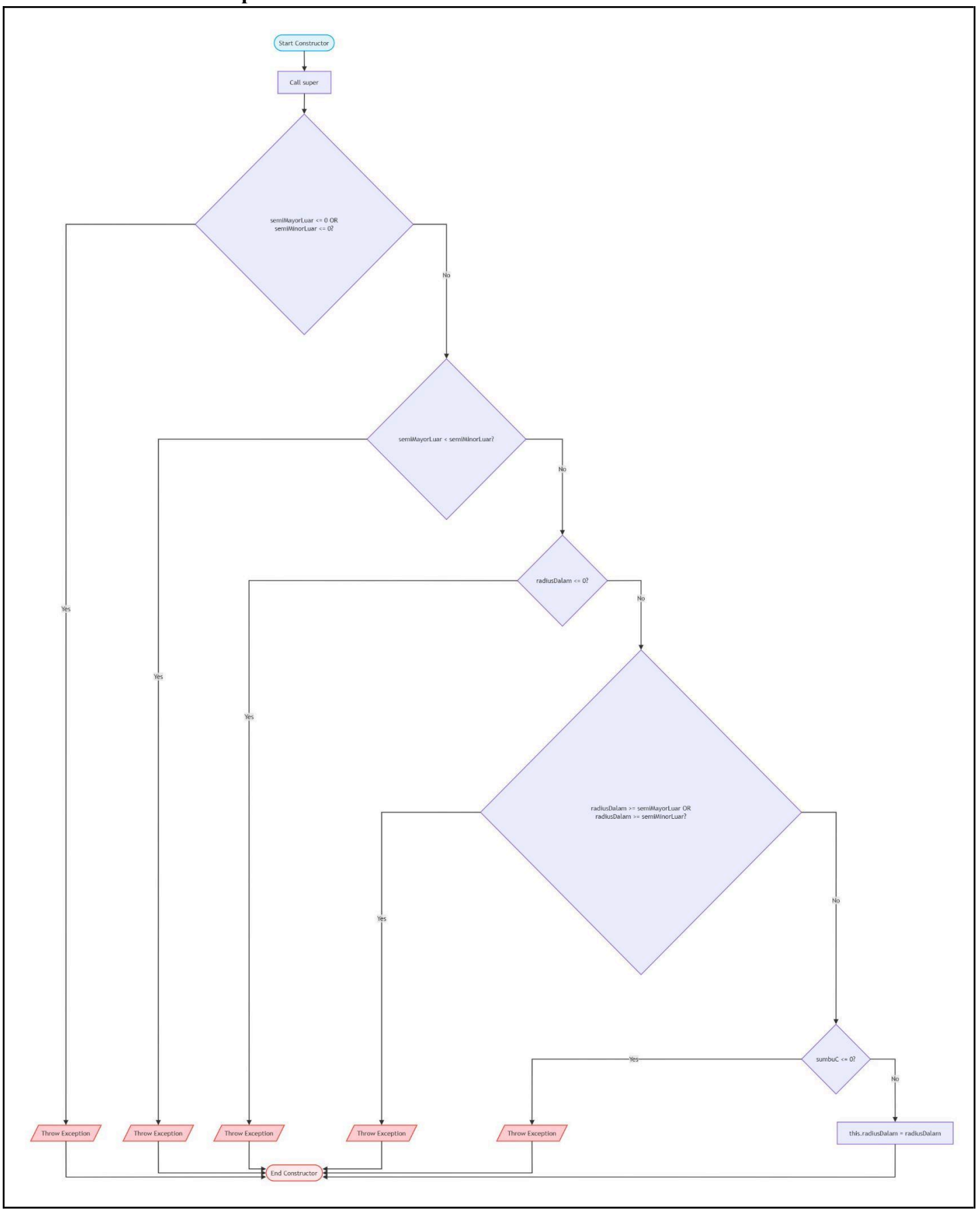
2.2 Flowchart BolaElips



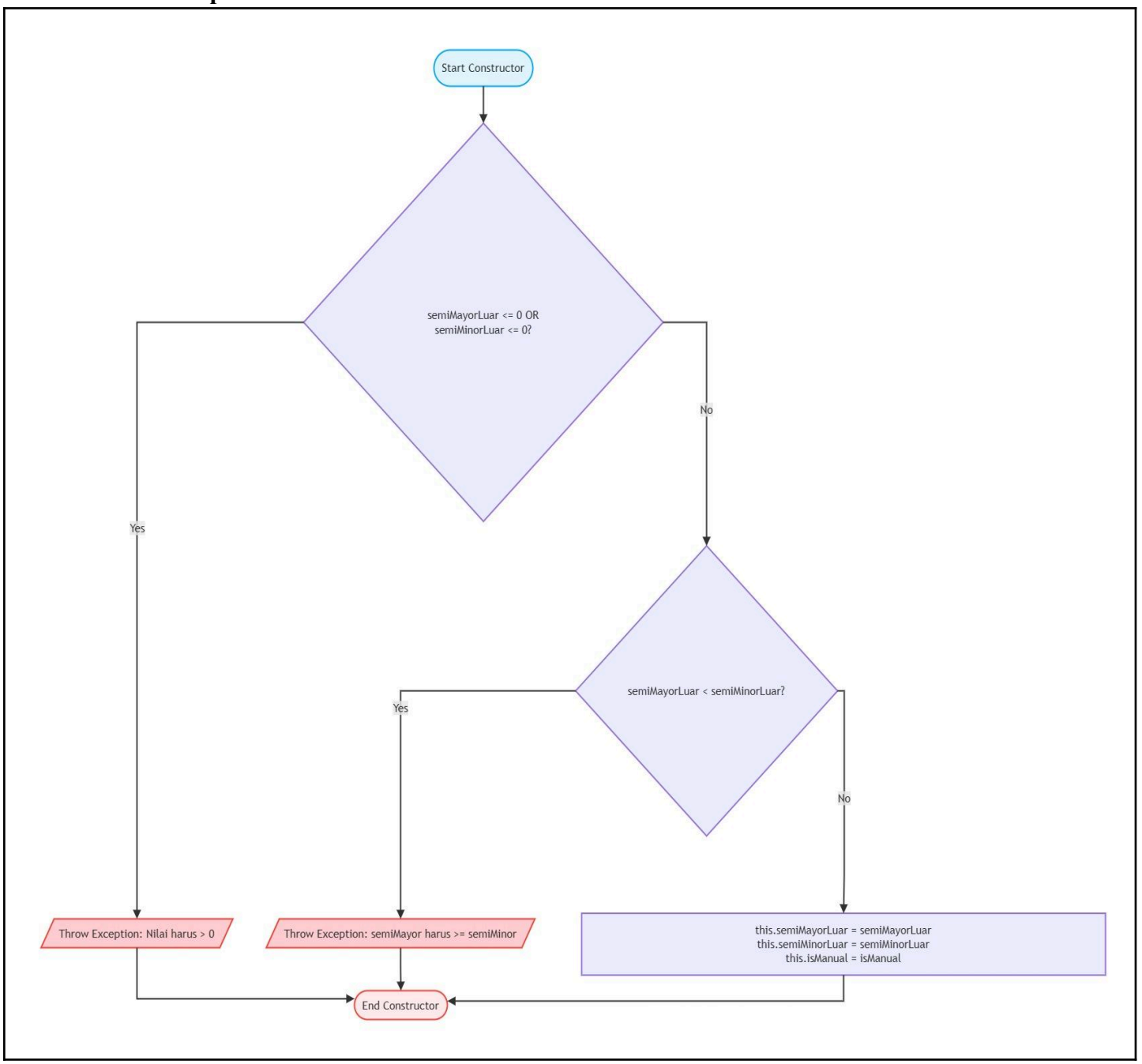
2.3 Flowchart CincinElips



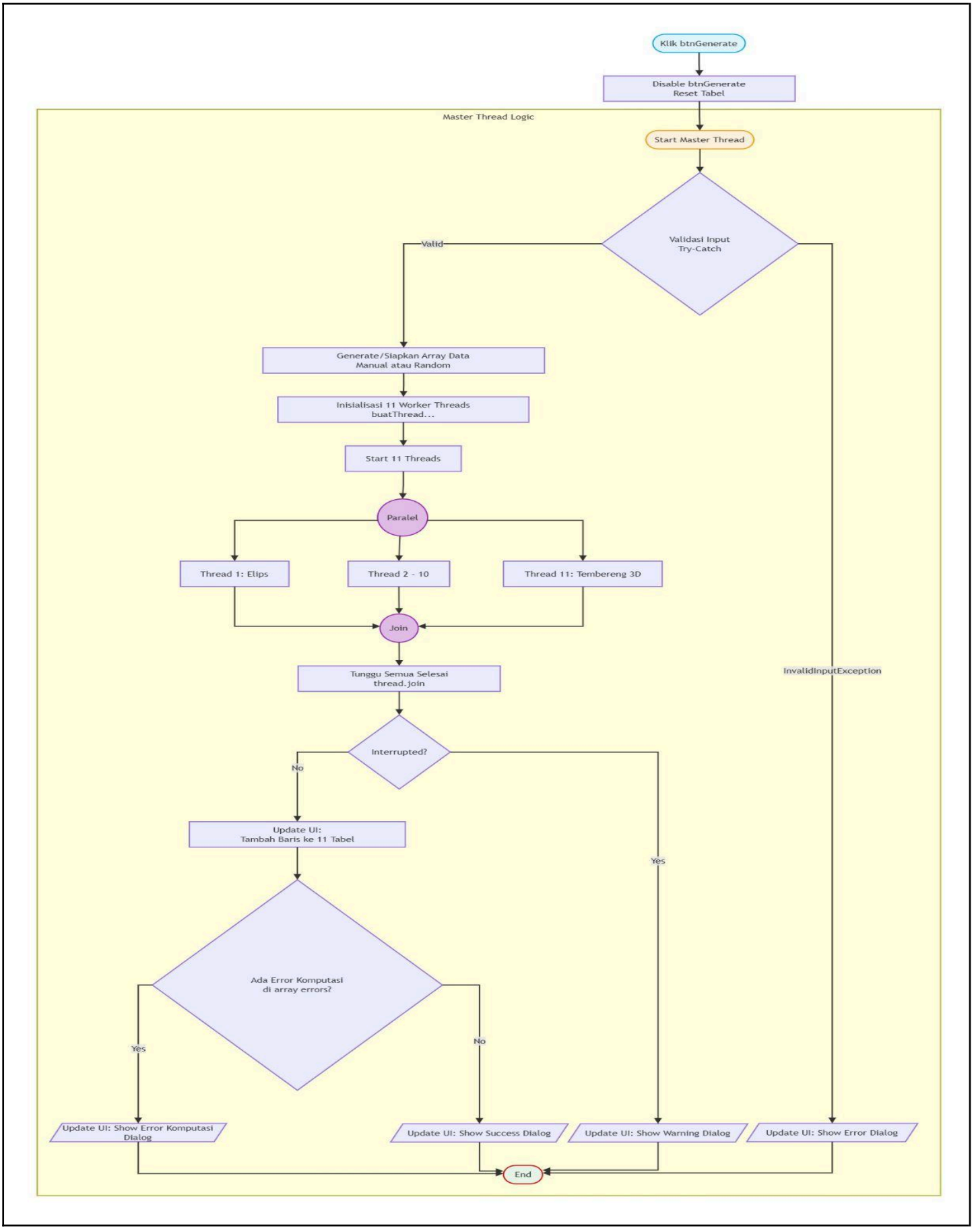
2.4 Flowchart CincinElips3D



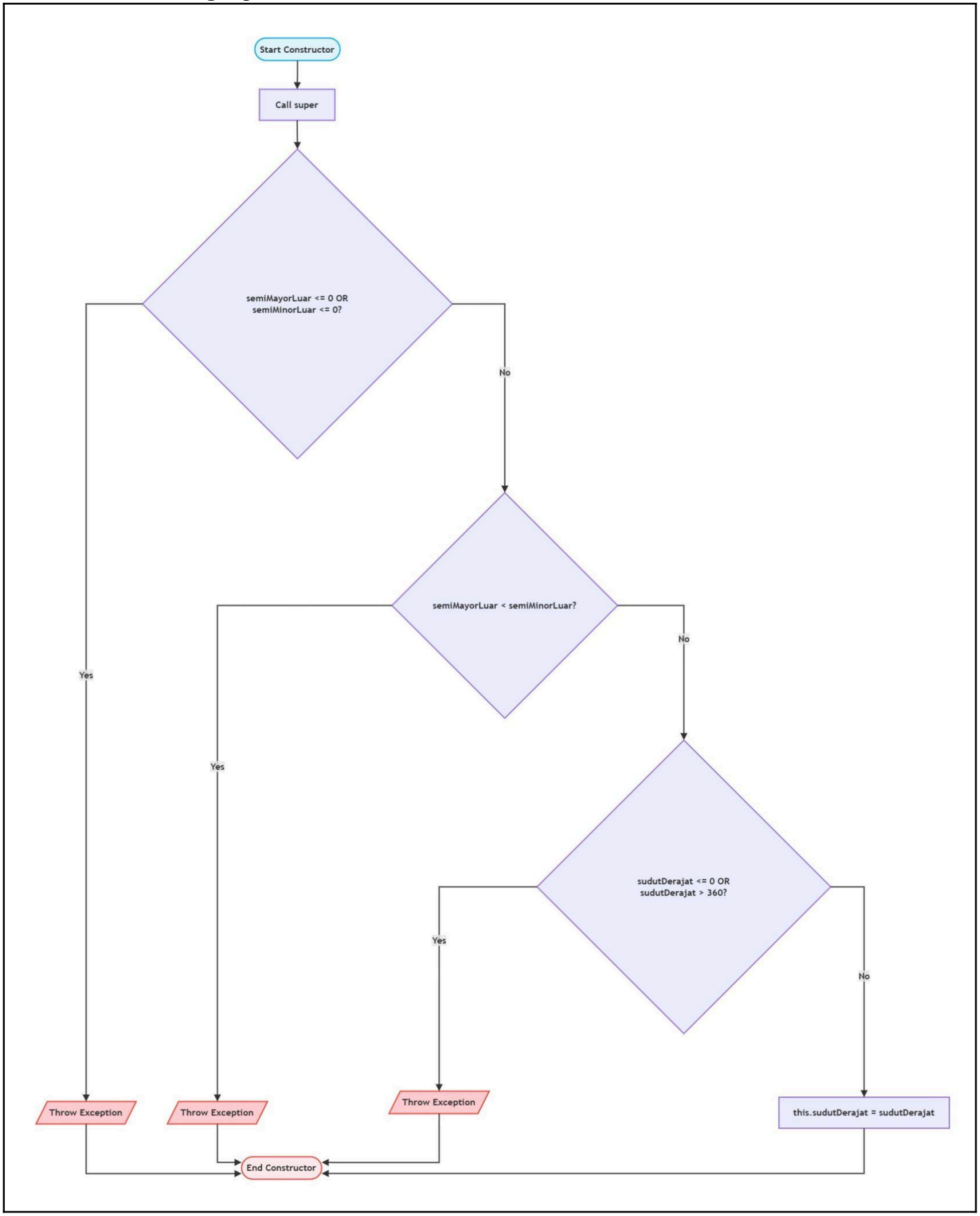
2.5 Flowchart Elips



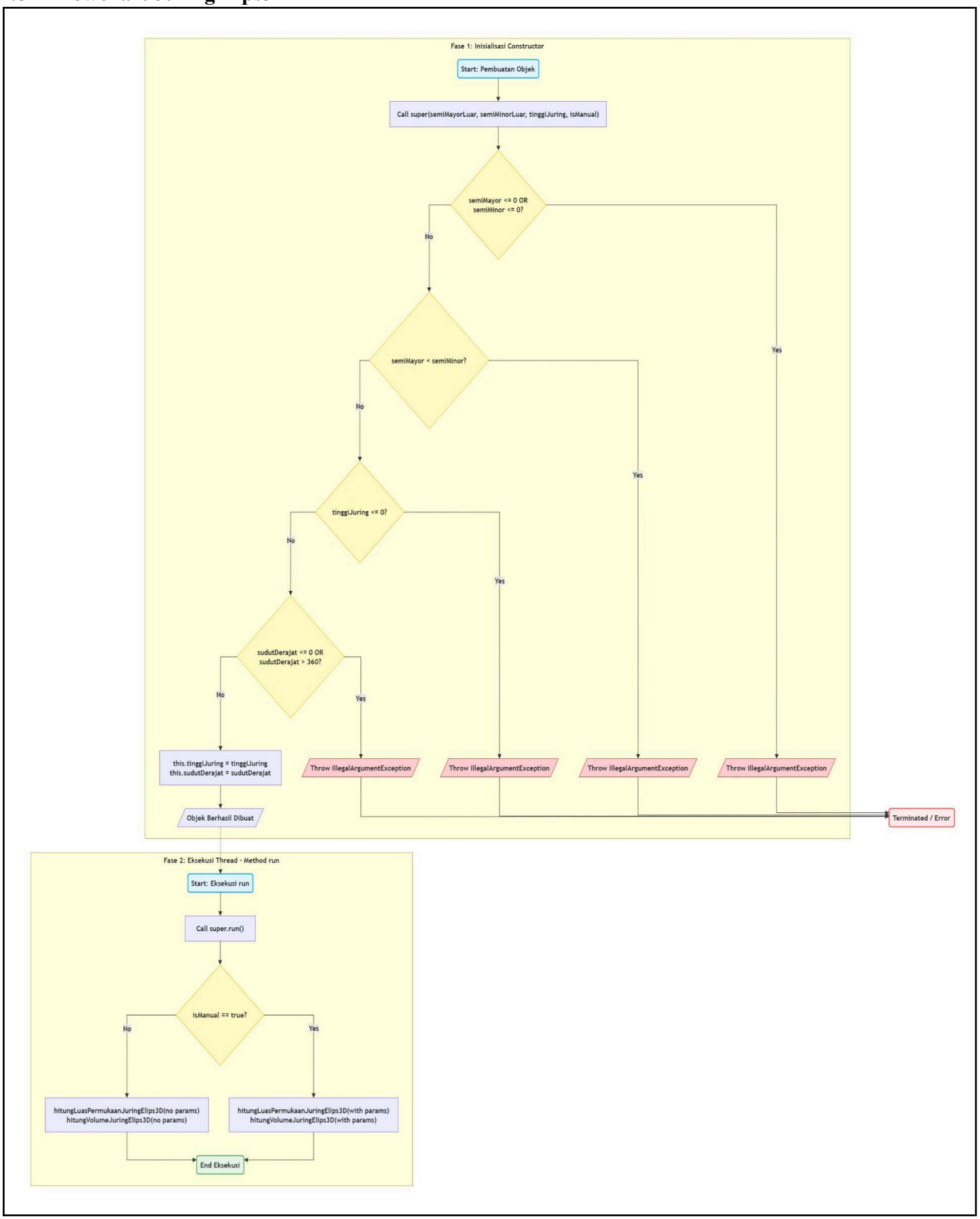
2.6 Flowchart GeometriGUI



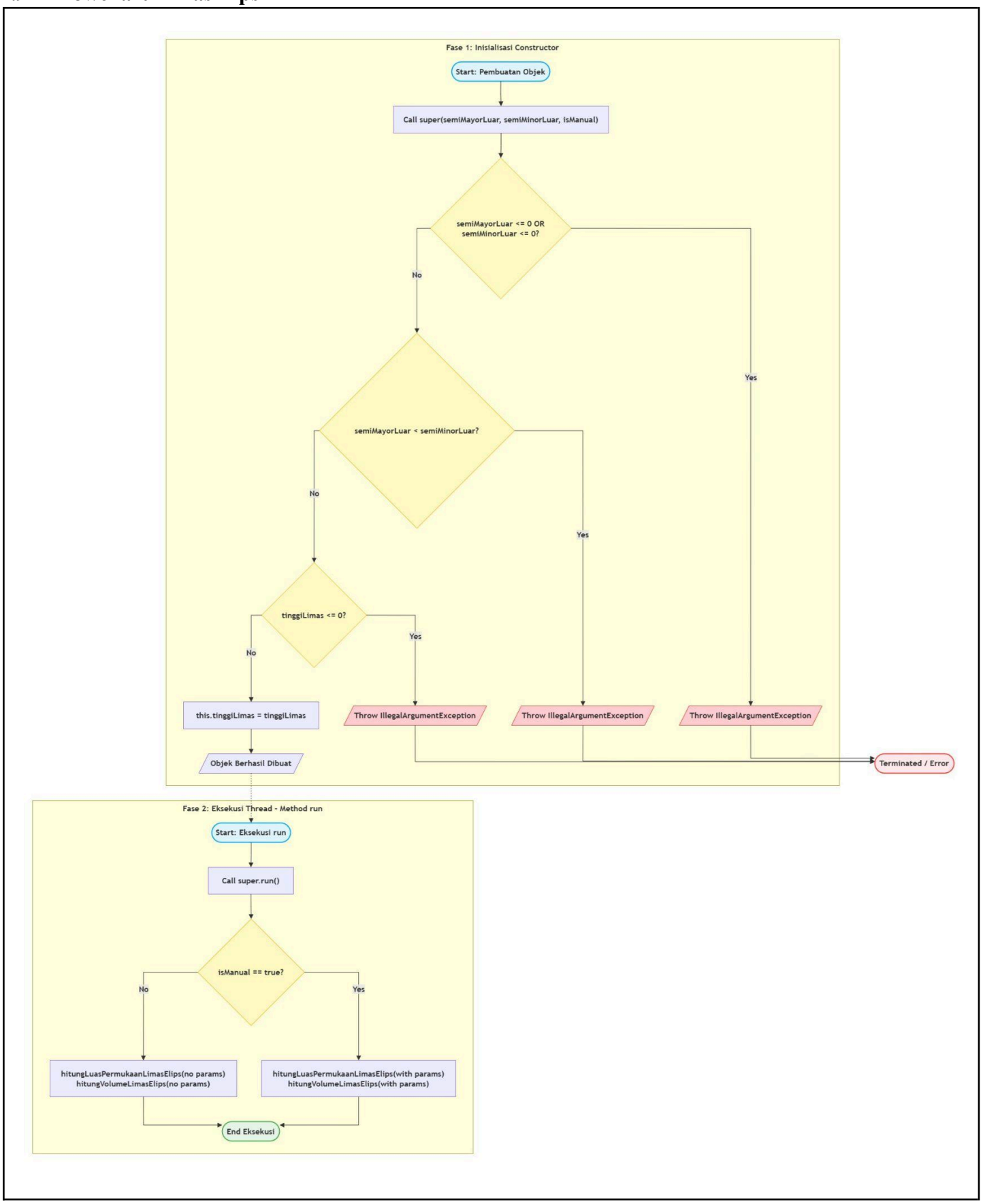
2.7 Flowchart JuringElips



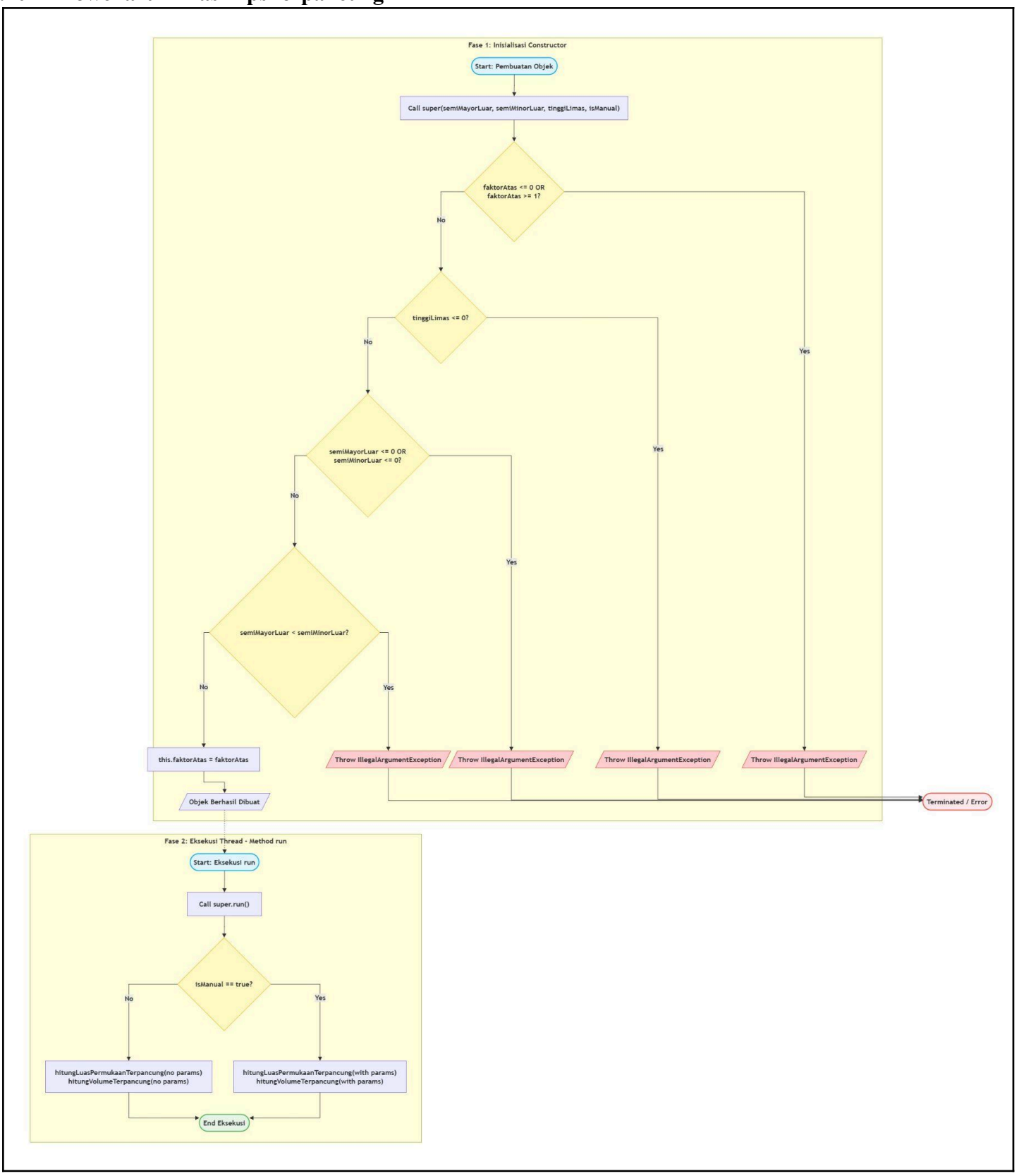
2.8 Flowchart JuringElips3D



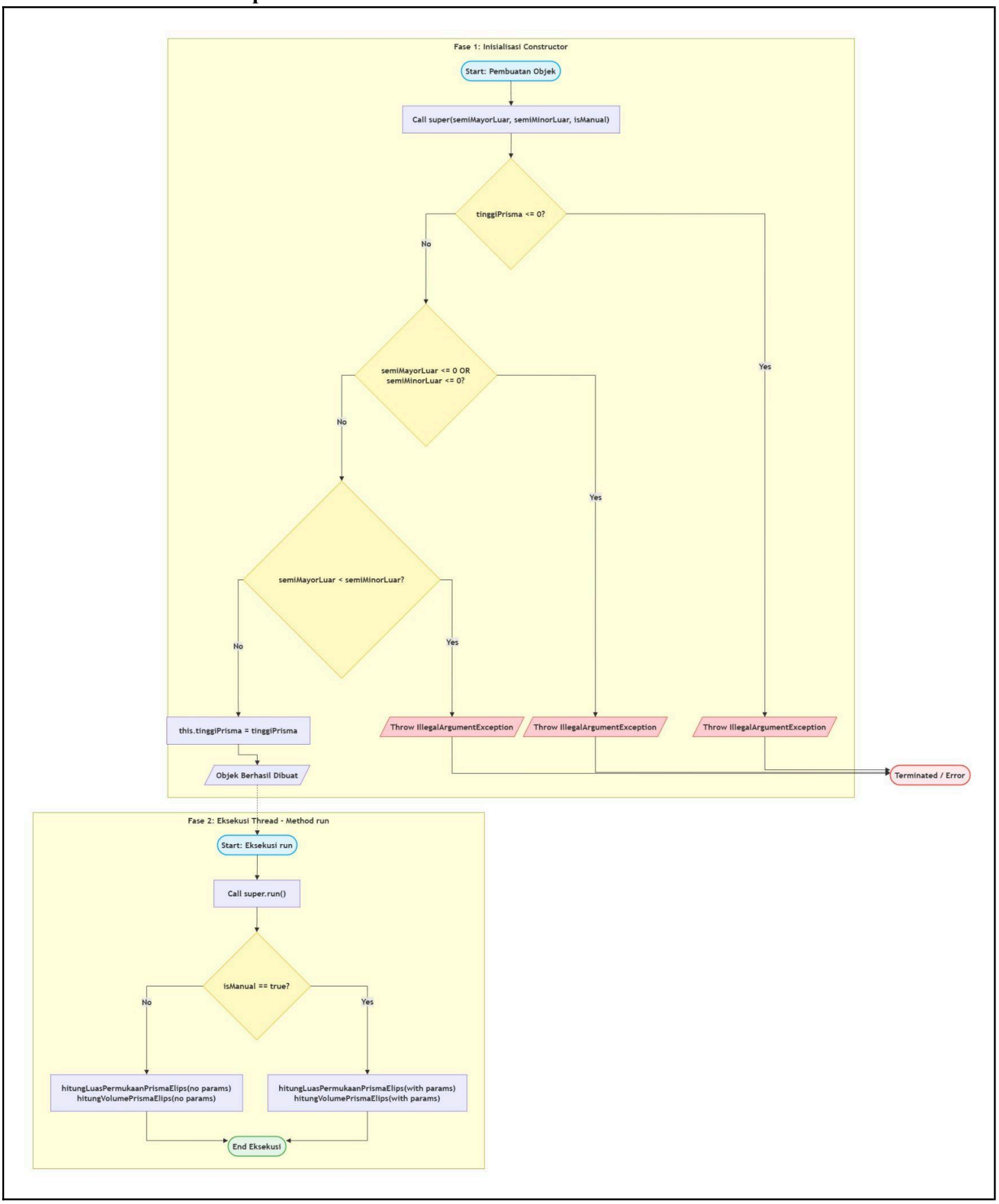
2.9 Flowchart LimasElips



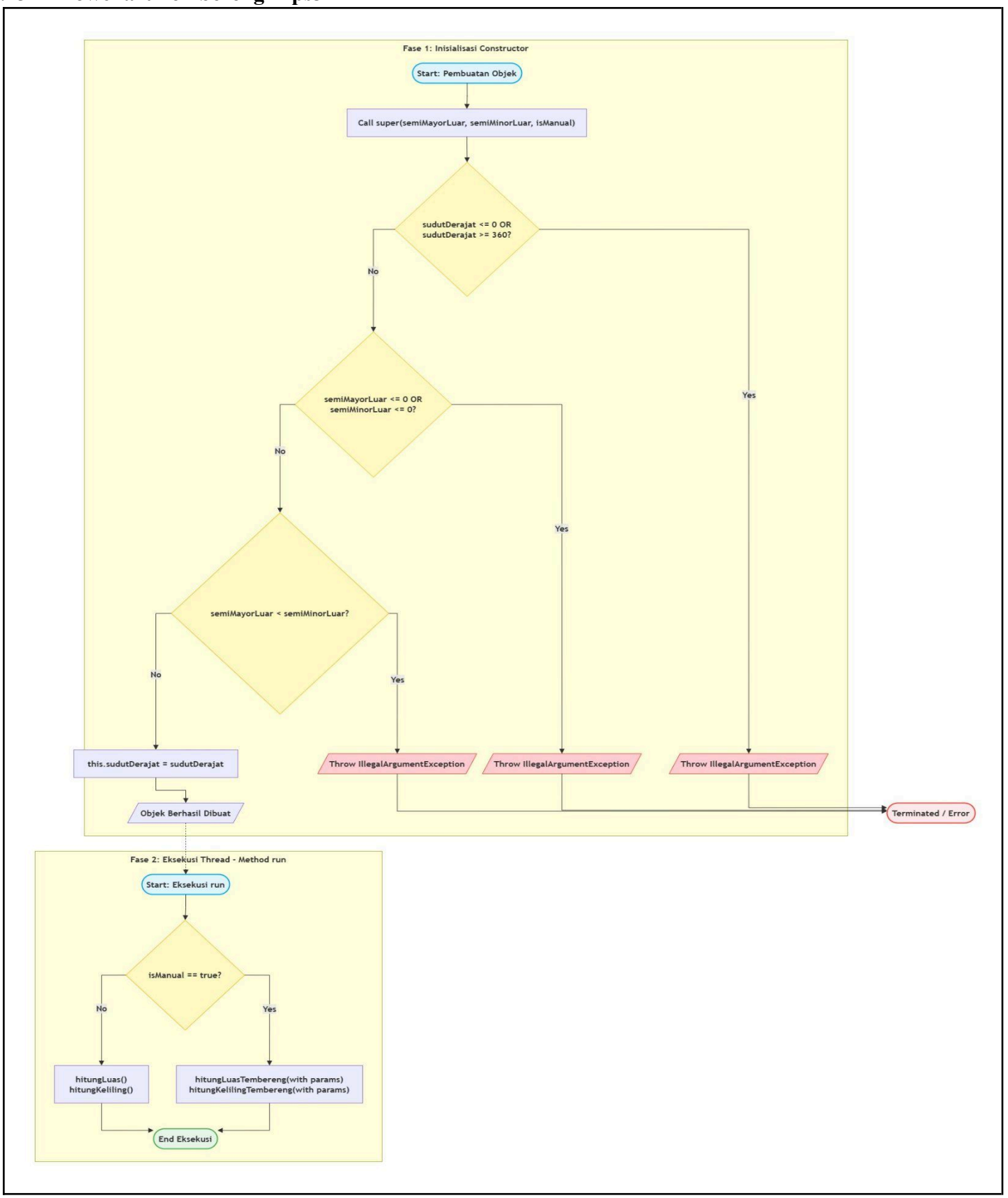
2.10 Flowchart LimasElipsTerpancung



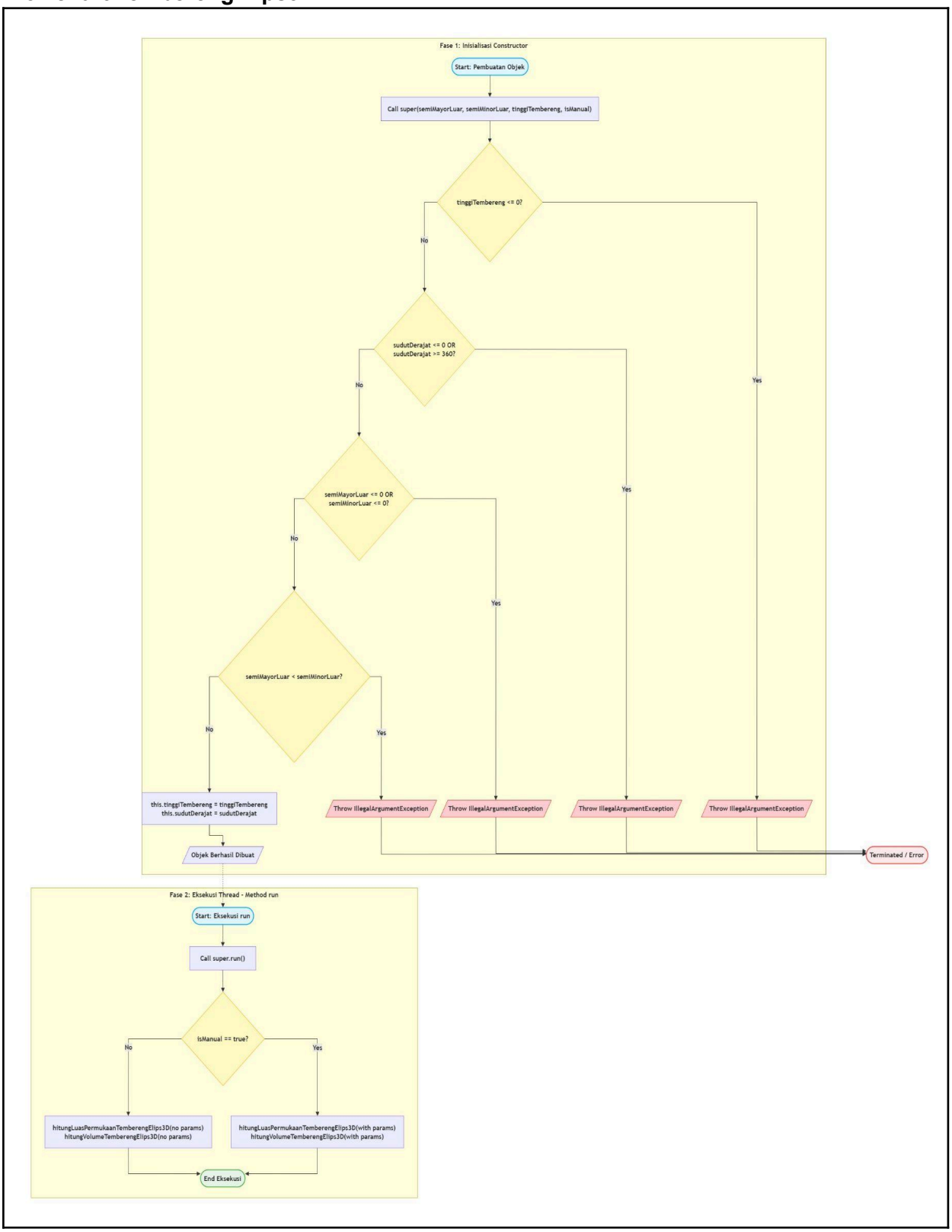
2.11 Flowchart PrismaElips



2.13 Flowchart TemberengElips3D



2.1 Flowchart TemberengElips3D



BAB III
IMPLEMENTASI PROGRAM

3.1 Class BangunGeometri.java

```
public interface BendaGeometri {
    public double hitungLuas();
    public double hitungKeliling();
    public double hitungLuas(double semiMayorLuar, double semiMinorLuar);
    public double hitungKeliling(double semiMayorLuar, double
semiMinorLuar);
}
```

3.2 Class Elips.java

```
public class Elips implements BendaGeometri, Runnable {

    // Atribut dibuat public mengikuti pola program acuan.
    public double semiMayorLuar, semiMinorLuar, luas, keliling;
    public boolean isManual;
    public final double PI = 3.14;

    public Elips(double semiMayorLuar, double semiMinorLuar, boolean
isManual) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Nilai Semi Mayor Luar dan
Semi Minor Luar elips harus lebih besar dari 0!");
        }

        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Semi Mayor Luar tidak boleh
lebih kecil dari Semi Minor Luar!");
        }
        this.semiMayorLuar = semiMayorLuar;
        this.semiMinorLuar = semiMinorLuar;
        this.isManual = isManual;
    }

    @Override
    public double hitungLuas() {
        this.luas = PI * this.semiMayorLuar * this.semiMinorLuar;
        return this.luas;
    }

    @Override
    public double hitungLuas(double semiMayorLuar, double semiMinorLuar)
throws IllegalArgumentException {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Input Semi Mayor Luar dan
```

```
Semi Minor Luar tidak boleh 0 atau negatif!");
        }

        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Semi Mayor Luar tidak boleh
lebih kecil dari Semi Minor Luar!");
        }

        this.luas = PI * semiMayorLuar * semiMinorLuar;
        return this.luas;
    }

    @Override
    public double hitungKeliling() {

        // Pendekatan Ramanujan untuk keliling elips.
        this.keliling = PI * (3 * (this.semiMayorLuar + this.semiMinorLuar)
                - Math.sqrt((3 * this.semiMayorLuar + this.semiMinorLuar) *
(this.semiMayorLuar + 3 * this.semiMinorLuar)));
        return this.keliling;
    }

    @Override
    public double hitungKeliling(double semiMayorLuar, double
semiMinorLuar) throws IllegalArgumentException {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Input Semi Mayor Luar dan
Semi Minor Luar tidak boleh 0 atau negatif!");
        }

        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Semi Mayor Luar tidak boleh
lebih kecil dari Semi Minor Luar!");
        }

        this.keliling = PI * (3 * (semiMayorLuar + semiMinorLuar)
                - Math.sqrt((3 * semiMayorLuar + semiMinorLuar) *
(semiMayorLuar + 3 * semiMinorLuar)));
        return this.keliling;
    }

    @Override
    public void run() {
        if (this.isManual) {
            this.hitungLuas(this.semiMayorLuar, this.semiMinorLuar);
            this.hitungKeliling(this.semiMayorLuar, this.semiMinorLuar);
        } else {
            this.hitungLuas();
            this.hitungKeliling();
        }
    }
}
```

3.3 Class BolaElips.java

```
public class BolaElips extends Elips implements Runnable {

    public double sumbuC, volumeBolaElips, luasPermukaanBolaElips,
luasAlas;
    public final double p = 1.6075;

    public BolaElips(double semiMayorLuar, double semiMinorLuar, double
sumbuC, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, isManual);
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Nilai semi mayor dan semi
minor bola elips harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Nilai semi mayor harus
lebih besar atau sama dengan semi minor bola elips!");
        }
        if (sumbuC <= 0) {
            throw new IllegalArgumentException("Nilai sumbu C bola elips
harus lebih besar dari 0!");
        }
        this.sumbuC = sumbuC;
    }

    public double hitungVolumeBolaElips() {
        if (super.luas <= 0) {
            throw new IllegalArgumentException("Nilai luas harus lebih
besar dari 0!");
        }
        this.volumeBolaElips = (4.0 / 3.0) * super.luas * this.sumbuC;
        return this.volumeBolaElips;
    }

    public double hitungVolumeBolaElips(double semiMayorLuar, double
semiMinorLuar, double sumbuC) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Nilai semi mayor dan semi
minor bola elips harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Nilai semi mayor harus
lebih besar atau sama dengan semi minor bola elips!");
        }
        if (sumbuC <= 0) {
            throw new IllegalArgumentException("Nilai sumbu C bola elips
harus lebih besar dari 0!");
        }
        luasAlas = super.hitungLuas(semiMayorLuar, semiMinorLuar);
```

```
        this.volumeBolaElips = (4.0 / 3.0) * luasAlas * sumbuC;
        return this.volumeBolaElips;
    }

    public double hitungLuasPermukaanBolaElips() {
        this.luasPermukaanBolaElips = 4 * Math.PI * Math.pow(
                (Math.pow(super.semiMayorLuar * super.semiMinorLuar, p)
                + Math.pow(super.semiMayorLuar * this.sumbuC, p)
                + Math.pow(super.semiMinorLuar * this.sumbuC, p)) / 3.0,
                1.0 / p
        );
        return this.luasPermukaanBolaElips;
    }

    public double hitungLuasPermukaanBolaElips(double semiMayorLuar, double
semiMinorLuar, double sumbuC) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Nilai semi mayor dan semi
minor bola elips harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Nilai semi mayor harus
lebih besar atau sama dengan semi minor bola elips!");
        }
        if (sumbuC <= 0) {
            throw new IllegalArgumentException("Nilai sumbu C bola elips
harus lebih besar dari 0!");
        }
        this.luasPermukaanBolaElips = 4 * Math.PI * Math.pow(
                (Math.pow(semiMayorLuar * semiMinorLuar, p)
                + Math.pow(semiMayorLuar * sumbuC, p)
                + Math.pow(semiMinorLuar * sumbuC, p)) / 3.0,
                1.0 / p
        );
        return this.luasPermukaanBolaElips;
    }

    @Override
    public void run() {
        super.run();
        if (this.isManual) {
            this.hitungLuasPermukaanBolaElips(semiMayorLuar, semiMinorLuar,
sumbuC);
            this.hitungVolumeBolaElips(semiMayorLuar, semiMinorLuar,
sumbuC);
        } else {
            this.hitungLuasPermukaanBolaElips();
            this.hitungVolumeBolaElips();
        }
    }
}
```

3.4 Class CincinElips.java

```
public class CincinElips extends Elips implements Runnable {

    public double semiMayorDalam, semiMinorDalam;
    public double luasCincin, kelilingCincin, luasElips, kelilingDalam,
kelilingElips, luasDalam;

    public CincinElips(double semiMayorLuar, double semiMinorLuar, double
semiMayorDalam, double semiMinorDalam, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, isManual);
        if (semiMayorDalam <= 0 || semiMinorDalam <= 0) {
            throw new IllegalArgumentException("Semi mayor dalam dan semi
minor dalam cincin elips harus lebih besar dari 0!");
        }
        if (semiMayorDalam >= semiMayorLuar || semiMinorDalam >=
semiMinorLuar) {
            throw new IllegalArgumentException("Elips dalam harus lebih
kecil dari elips luar!");
        }
        if (semiMayorDalam < semiMinorDalam) {
            throw new IllegalArgumentException("Semi mayor dalam tidak
boleh lebih kecil dari semi minor dalam!");
        }
        this.semiMayorDalam = semiMayorDalam;
        this.semiMinorDalam = semiMinorDalam;
    }

    @Override
    public double hitungLuas() {
        return this.hitungLuasCincin();
    }

    @Override
    public double hitungKeliling() {
        return this.hitungKelilingCincin();
    }

    public double hitungLuasCincin() {
        this.luasElips = super.hitungLuas(super.semiMayorLuar,
super.semiMinorLuar);
        this.luasDalam = super.hitungLuas(this.semiMayorDalam,
this.semiMinorDalam);
        this.luasCincin = this.luasElips - this.luasDalam;
        return this.luasCincin;
    }

    public double hitungLuasCincin(double semiMayorLuar, double
semiMinorLuar, double semiMayorDalam, double semiMinorDalam) {
        if (semiMayorDalam <= 0 || semiMinorDalam <= 0) {
            throw new IllegalArgumentException("Semi mayor dalam dan semi
```

```
minor dalam cincin elips harus lebih besar dari 0!");
        }
        if (semiMayorDalam >= semiMayorLuar || semiMinorDalam >=
semiMinorLuar) {
            throw new IllegalArgumentException("Elips dalam harus lebih
kecil dari elips luar!");
        }
        if (semiMayorDalam < semiMinorDalam) {
            throw new IllegalArgumentException("Semi mayor dalam tidak
boleh lebih kecil dari semi minor dalam!");
        }
        this.luasElips = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        this.luasDalam = super.hitungLuas(semiMayorDalam, semiMinorDalam);
        super.luas = this.luasElips;
        this.luasCincin = this.luasElips - this.luasDalam;
        return this.luasCincin;
    }

    public double hitungKelilingCincin() {
        this.kelilingElips = super.hitungKeliling(super.semiMayorLuar,
super.semiMinorLuar);
        this.kelilingDalam = super.hitungKeliling(this.semiMayorDalam,
this.semiMinorDalam);
        super.keliling = this.kelilingElips;
        this.kelilingCincin = this.kelilingElips + this.kelilingDalam;
        return this.kelilingCincin;
    }

    public double hitungKelilingCincin(double semiMayorLuar, double
semiMinorLuar, double semiMayorDalam, double semiMinorDalam) {
        if (semiMayorDalam <= 0 || semiMinorDalam <= 0) {
            throw new IllegalArgumentException("Semi mayor dalam dan semi
minor dalam cincin elips harus lebih besar dari 0!");
        }
        if (semiMayorDalam >= semiMayorLuar || semiMinorDalam >=
semiMinorLuar) {
            throw new IllegalArgumentException("Elips dalam harus lebih
kecil dari elips luar!");
        }
        if (semiMayorDalam < semiMinorDalam) {
            throw new IllegalArgumentException("Semi mayor dalam tidak
boleh lebih kecil dari semi minor dalam!");
        }
        this.kelilingElips = super.hitungKeliling(semiMayorLuar,
semiMinorLuar);
        this.kelilingDalam = super.hitungKeliling(semiMayorDalam,
semiMinorDalam);
        super.keliling = this.kelilingElips;
        this.kelilingCincin = this.kelilingElips + this.kelilingDalam;
        return this.kelilingCincin;
    }
```

```
    @Override
    public void run() {
        if (this.isManual) {
            this.hitungLuasCincin(semiMayorLuar, semiMinorLuar,
semiMayorDalam, semiMinorDalam);
            this.hitungKelilingCincin(semiMayorLuar, semiMinorLuar,
semiMayorDalam, semiMinorDalam);
        } else {
            this.hitungLuas();
            this.hitungKeliling();
        }
    }
}
```

3.5 Class JuringElips.java

```
public class JuringElips extends Elips implements Runnable {

    public double sudutDerajat, luasJuring, kelilingJuring, luasElips,
kelilingElips, panjangBusur, rataRataJariJari, panjangBusurElips;
    public double sisiAwal, sisiAkhir, theta, h, total, integrandBusur,
jarakPusatKeTitikElips, jumlahSegmen;

    public JuringElips(double semiMayorLuar, double semiMinorLuar, double
sudutDerajat, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, isManual);
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Semi mayor luar dan semi
minor luar juring elips harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Semi mayor luar tidak boleh
lebih kecil dari semi minor luar!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        this.sudutDerajat = sudutDerajat;
    }

    @Override
    public double hitungLuas() {
        return this.hitungLuasJuring();
    }

    @Override
    public double hitungKeliling() {
        return this.hitungKelilingJuring();
    }
```

```
    public double hitungLuasJuring() {
        this.luasJuring = (this.sudutDerajat / 360.0) * super.luas;
        return this.luasJuring;
    }

    public double hitungLuasJuring(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Semi mayor luar dan semi
minor luar juring elips harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Semi mayor luar tidak boleh
lebih kecil dari semi minor luar!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        this.luasElips = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        this.luasJuring = (sudutDerajat / 360.0) * this.luasElips;
        return this.luasJuring;
    }

    public double hitungKelilingJuring() {
        this.panjangBusur = hitungPanjangBusurElips(super.semiMayorLuar,
super.semiMinorLuar, this.sudutDerajat);
        if (Math.abs(this.sudutDerajat - 360.0) < 0.0000001) {
            this.sisiAwal = 0;
            this.sisiAkhir = 0;
        } else {
            this.sisiAwal = super.semiMayorLuar;
            this.sisiAkhir =
hitungJarakPusatKeTitikElips(super.semiMayorLuar, super.semiMinorLuar,
this.sudutDerajat);
        }
        this.rataRataJariJari = (this.sisiAwal + this.sisiAkhir) / 2.0;
        this.kelilingJuring = this.panjangBusur + this.sisiAwal +
this.sisiAkhir;
        return this.kelilingJuring;
    }

    public double hitungKelilingJuring(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Semi mayor luar dan semi
minor luar juring elips harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Semi mayor luar tidak boleh
lebih kecil dari semi minor luar!");
```

```
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        this.kelilingElips = super.hitungKeliling(semiMayorLuar,
semiMinorLuar);
        this.panjangBusur = hitungPanjangBusurElips(semiMayorLuar,
semiMinorLuar, sudutDerajat);
        if (Math.abs(sudutDerajat - 360.0) < 0.0000001) {
            this.sisiAwal = 0;
            this.sisiAkhir = 0;
        } else {
            this.sisiAwal = semiMayorLuar;
            this.sisiAkhir = hitungJarakPusatKeTitikElips(semiMayorLuar,
semiMinorLuar, sudutDerajat);
        }
        this.rataRataJariJari = (this.sisiAwal + this.sisiAkhir) / 2.0;
        this.kelilingJuring = this.panjangBusur + this.sisiAwal +
this.sisiAkhir;
        return this.kelilingJuring;
    }

    public double hitungPanjangBusurElips(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Semi mayor luar dan semi
minor luar juring elips harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Semi mayor luar tidak boleh
lebih kecil dari semi minor luar!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        theta = Math.toRadians(sudutDerajat);
        jumlahSegmen = 1000;
        this.h = theta / jumlahSegmen;
        this.total = integrandBusur(semiMayorLuar, semiMinorLuar, 0) +
integrandBusur(semiMayorLuar, semiMinorLuar, theta);

        for (int i = 1; i < jumlahSegmen; i++) {
            double t = i * this.h;
            this.total += (i % 2 == 0 ? 2 : 4) *
integrandBusur(semiMayorLuar, semiMinorLuar, t);
        }

        panjangBusurElips = (this.h / 3.0) * this.total;
        return panjangBusurElips;
    }
```

```
    public double integrandBusur(double semiMayorLuar, double
semiMinorLuar, double t) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Semi mayor luar dan semi
minor luar juring elips harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Semi mayor luar tidak boleh
lebih kecil dari semi minor luar!");
        }

        integrandBusur = Math.sqrt(Math.pow(-semiMayorLuar * Math.sin(t),
2) + Math.pow(semiMinorLuar * Math.cos(t), 2));

        return integrandBusur;
    }

    public double hitungJarakPusatKeTitikElips(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Semi mayor luar dan semi
minor luar juring elips harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Semi mayor luar tidak boleh
lebih kecil dari semi minor luar!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        theta = Math.toRadians(sudutDerajat);
        jarakPusatKeTitikElips = Math.sqrt(
                Math.pow(semiMayorLuar * Math.cos(theta), 2)
                + Math.pow(semiMinorLuar * Math.sin(theta), 2)
        );
        return jarakPusatKeTitikElips;
    }

    @Override
    public void run() {
        if (this.isManual) {
            this.hitungLuasJuring(semiMayorLuar, semiMinorLuar,
sudutDerajat);
            this.hitungKelilingJuring(semiMayorLuar, semiMinorLuar,
sudutDerajat);
        } else {
            this.hitungLuas();
            this.hitungKeliling();
        }
    }
```

```
}
```

3.6 Class LimasElips.java

```
public class LimasElips extends Elips implements Runnable {

    public double tinggiLimas, volumeLimasElips, luasPermukaanLimasElips,
luasSelimutLimasElips;
    public double luasAlas, garisPelukisA, garisPelukisB, rataGariPelukis,
rataGarisPelukis, kelilingAlas;

    public LimasElips(double semiMayorLuar, double semiMinorLuar, double
tinggiLimas, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, isManual);
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        if (tinggiLimas <= 0) {
            throw new IllegalArgumentException("Tinggi limas/kerucut elips
harus lebih besar dari 0!");
        }
        this.tinggiLimas = tinggiLimas;
    }

    public double hitungVolumeLimasElips() {
        this.volumeLimasElips = (1.0 / 3.0) * super.luas *
this.tinggiLimas;
        return this.volumeLimasElips;
    }

    public double hitungVolumeLimasElips(double semiMayorLuar, double
semiMinorLuar, double tinggiLimas) {
        if (tinggiLimas <= 0) {
            throw new IllegalArgumentException("Tinggi limas/kerucut elips
harus lebih besar dari 0!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        luasAlas = super.hitungLuas(semiMayorLuar, semiMinorLuar);
```

```
        this.volumeLimasElips = (1.0 / 3.0) * luasAlas * tinggiLimas;
        return this.volumeLimasElips;
    }

    public double hitungLuasPermukaanLimasElips() {
        garisPelukisA = Math.sqrt(Math.pow(super.semiMayorLuar, 2) +
Math.pow(this.tinggiLimas, 2));
        garisPelukisB = Math.sqrt(Math.pow(super.semiMinorLuar, 2) +
Math.pow(this.tinggiLimas, 2));
        rataGarisPelukis = (garisPelukisA + garisPelukisB) / 2.0;
        this.luasSelimutLimasElips = 0.5 * super.keliling *
rataGarisPelukis;
        this.luasPermukaanLimasElips = super.luas +
this.luasSelimutLimasElips;
        return this.luasPermukaanLimasElips;
    }

    public double hitungLuasPermukaanLimasElips(double semiMayorLuar,
double semiMinorLuar, double tinggiLimas) {
        if (tinggiLimas <= 0) {
            throw new IllegalArgumentException("Tinggi limas/kerucut elips
harus lebih besar dari 0!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        luasAlas = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        kelilingAlas = super.hitungKeliling(semiMayorLuar, semiMinorLuar);
        garisPelukisA = Math.sqrt(Math.pow(semiMayorLuar, 2) +
Math.pow(tinggiLimas, 2));
        garisPelukisB = Math.sqrt(Math.pow(semiMinorLuar, 2) +
Math.pow(tinggiLimas, 2));
        rataGarisPelukis = (garisPelukisA + garisPelukisB) / 2.0;
        this.luasSelimutLimasElips = 0.5 * kelilingAlas * rataGarisPelukis;
        this.luasPermukaanLimasElips = luasAlas +
this.luasSelimutLimasElips;
        return this.luasPermukaanLimasElips;
    }

    @Override
    public void run() {
        super.run();
        if (this.isManual) {
            this.hitungLuasPermukaanLimasElips(semiMayorLuar,
semiMinorLuar, tinggiLimas);
            this.hitungVolumeLimasElips(semiMayorLuar, semiMinorLuar,
tinggiLimas);
```

```
        } else {
            this.hitungLuasPermukaanLimasElips();
            this.hitungVolumeLimasElips();
        }
    }
}
```

3.7 Class LimasElipsTerpancung.java

```
public class LimasElipsTerpancung extends LimasElips implements Runnable {

    public double faktorAtas, luasAtas, kelilingAtas, volumeTerpancung,
luasPermukaanTerpancung, luasSelimutTerpancung;
    public double luasAlas, garisPelukisA, garisPelukisB, rataGariPelukis,
rataGarisPelukis, selisihA, selisihB, kelilingAlas;

    public LimasElipsTerpancung(double semiMayorLuar, double semiMinorLuar,
double tinggiLimas, double faktorAtas, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, tinggiLimas, isManual);
        if (faktorAtas <= 0 || faktorAtas >= 1) {
            throw new IllegalArgumentException("Faktor atas limas elips
terpancung harus lebih dari 0 dan kurang dari 1!");
        }
        if (tinggiLimas <= 0) {
            throw new IllegalArgumentException("Tinggi limas elips
terpancung harus lebih dari 0!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        this.faktorAtas = faktorAtas;
    }

    public double hitungVolumeTerpancung() {
        this.luasAtas = super.luas * Math.pow(this.faktorAtas, 2);
        this.volumeTerpancung = (this.tinggiLimas / 3.0) * (super.luas +
this.luasAtas + Math.sqrt(super.luas * this.luasAtas));
        return this.volumeTerpancung;
    }

    public double hitungVolumeTerpancung(double semiMayorLuar, double
semiMinorLuar, double tinggiLimas, double faktorAtas) {
        if (faktorAtas <= 0 || faktorAtas >= 1) {
            throw new IllegalArgumentException("Faktor atas limas elips
terpancung harus lebih dari 0 dan kurang dari 1!");
```

```
        }
        if (tinggiLimas <= 0) {
            throw new IllegalArgumentException("Tinggi limas elips
terpancung harus lebih dari 0!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        this.luasAlas = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        this.luasAtas = luasAlas * Math.pow(faktorAtas, 2);
        this.volumeTerpancung = (tinggiLimas / 3.0) * (luasAlas +
this.luasAtas + Math.sqrt(luasAlas * this.luasAtas));
        return this.volumeTerpancung;
    }

    public double hitungLuasPermukaanTerpancung() {
        this.luasAtas = super.luas * Math.pow(this.faktorAtas, 2);
        this.kelilingAtas = super.keliling * this.faktorAtas;

        selisihA = super.semiMayorLuar - (super.semiMayorLuar *
this.faktorAtas);
        selisihB = super.semiMinorLuar - (super.semiMinorLuar *
this.faktorAtas);
        this.garisPelukisA = Math.sqrt(Math.pow(selisihA, 2) +
Math.pow(this.tinggiLimas, 2));
        this.garisPelukisB = Math.sqrt(Math.pow(selisihB, 2) +
Math.pow(this.tinggiLimas, 2));
        this.rataGarisPelukis = (garisPelukisA + garisPelukisB) / 2.0;

        this.luasSelimutTerpancung = 0.5 * (super.keliling +
this.kelilingAtas) * rataGarisPelukis;
        this.luasPermukaanTerpancung = super.luas + this.luasAtas +
this.luasSelimutTerpancung;
        return this.luasPermukaanTerpancung;
    }

    public double hitungLuasPermukaanTerpancung(double semiMayorLuar,
double semiMinorLuar, double tinggiLimas, double faktorAtas) {
        if (tinggiLimas <= 0) {
            throw new IllegalArgumentException("Tinggi limas elips
terpancung harus lebih dari 0!");
        }
        if (faktorAtas <= 0 || faktorAtas >= 1) {
            throw new IllegalArgumentException("Faktor atas limas elips
terpancung harus lebih dari 0 dan kurang dari 1!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
```

```
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        this.luasAlas = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        this.kelilingAlas = super.hitungKeliling(semiMayorLuar,
semiMinorLuar);
        this.luasAtas = luasAlas * Math.pow(faktorAtas, 2);
        this.kelilingAtas = kelilingAlas * faktorAtas;

        selisihA = semiMayorLuar - (semiMayorLuar * faktorAtas);
        selisihB = semiMinorLuar - (semiMinorLuar * faktorAtas);
        this.garisPelukisA = Math.sqrt(Math.pow(selisihA, 2) +
Math.pow(tinggiLimas, 2));
        this.garisPelukisB = Math.sqrt(Math.pow(selisihB, 2) +
Math.pow(tinggiLimas, 2));
        this.rataGarisPelukis = (garisPelukisA + garisPelukisB) / 2.0;

        this.luasSelimutTerpancung = 0.5 * (kelilingAlas +
this.kelilingAtas) * rataGarisPelukis;
        this.luasPermukaanTerpancung = luasAlas + this.luasAtas +
this.luasSelimutTerpancung;
        return this.luasPermukaanTerpancung;
    }

    @Override
    public void run() {
        super.run();
        if (this.isManual) {
            this.hitungLuasPermukaanTerpancung(semiMayorLuar,
semiMinorLuar, tinggiLimas, faktorAtas);
            this.hitungVolumeTerpancung(semiMayorLuar, semiMinorLuar,
tinggiLimas, faktorAtas);
        } else {
            this.hitungLuasPermukaanTerpancung();
            this.hitungVolumeTerpancung();
        }
    }
}
```

3.8 Class PrismaElips.java

```
public class PrismaElips extends Elips implements Runnable {

    public double tinggiPrisma, volumePrismaElips,
luasPermukaanPrismaElips;
    public double luasAlas, kelilingAlas;

    public PrismaElips(double semiMayorLuar, double semiMinorLuar, double
tinggiPrisma, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, isManual);
        if (tinggiPrisma <= 0) {
            throw new IllegalArgumentException("Tinggi prisma/tabung elips
harus lebih besar dari 0!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        this.tinggiPrisma = tinggiPrisma;
    }

    public double hitungVolumePrismaElips() {
        this.volumePrismaElips = super.luas * this.tinggiPrisma;
        return this.volumePrismaElips;
    }

    public double hitungVolumePrismaElips(double semiMayorLuar, double
semiMinorLuar, double tinggiPrisma) {
        if (tinggiPrisma <= 0) {
            throw new IllegalArgumentException("Tinggi prisma/tabung elips
harus lebih besar dari 0!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        this.luasAlas = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        this.volumePrismaElips = luasAlas * tinggiPrisma;
        return this.volumePrismaElips;
    }

    public double hitungLuasPermukaanPrismaElips() {
```

```
        this.luasPermukaanPrismaElips = (2 * super.luas) + (super.keliling
* this.tinggiPrisma);
        return this.luasPermukaanPrismaElips;
    }

    public double hitungLuasPermukaanPrismaElips(double semiMayorLuar,
double semiMinorLuar, double tinggiPrisma) {
        if (tinggiPrisma <= 0) {
            throw new IllegalArgumentException("Tinggi prisma/tabung elips
harus lebih besar dari 0!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        this.luasAlas = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        this.kelilingAlas = super.hitungKeliling(semiMayorLuar,
semiMinorLuar);
        this.luasPermukaanPrismaElips = (2 * luasAlas) + (kelilingAlas *
tinggiPrisma);
        return this.luasPermukaanPrismaElips;
    }

    @Override
    public void run() {
        super.run();
        if (this.isManual) {
            this.hitungLuasPermukaanPrismaElips(semiMayorLuar,
semiMinorLuar, tinggiPrisma);
            this.hitungVolumePrismaElips(semiMayorLuar, semiMinorLuar,
tinggiPrisma);
        } else {
            this.hitungLuasPermukaanPrismaElips();
            this.hitungVolumePrismaElips();
        }
    }
}
```

3.9 Class TemberengElips.java

```
public class TemberengElips extends Elips implements Runnable {

    public double sudutDerajat, luasTembereng, kelilingTembereng;
    public double theta, panjangBusur, taliBusur, kelilingElips,
integrandBusur, jumlahSegmen, h, total;
```

```
    public TemberengElips(double semiMayorLuar, double semiMinorLuar,
double sudutDerajat, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, isManual);
        if (sudutDerajat <= 0 || sudutDerajat >= 360) {
            throw new IllegalArgumentException("Sudut tembereng elips harus
lebih dari 0 dan kurang dari 360 derajat!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }

        this.sudutDerajat = sudutDerajat;
    }

    @Override
    public double hitungLuas() {
        return this.hitungLuasTembereng();
    }

    @Override
    public double hitungKeliling() {
        return this.hitungKelilingTembereng();
    }

    public double hitungLuasTembereng() {
        theta = Math.toRadians(this.sudutDerajat);
        this.luasTembereng = 0.5 * super.semiMayorLuar *
super.semiMinorLuar * (theta - Math.sin(theta));
        return this.luasTembereng;
    }

    public double hitungLuasTembereng(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        if (sudutDerajat <= 0 || sudutDerajat >= 360) {
            throw new IllegalArgumentException("Sudut tembereng elips harus
lebih dari 0 dan kurang dari 360 derajat!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        theta = Math.toRadians(sudutDerajat);
        this.luasTembereng = 0.5 * semiMayorLuar * semiMinorLuar * (theta -
```

```
Math.sin(theta));
        return this.luasTembereng;
    }

    public double hitungKelilingTembereng() {
        this.theta = Math.toRadians(this.sudutDerajat);
        this.panjangBusur = hitungPanjangBusurElips(super.semiMayorLuar,
super.semiMinorLuar, this.sudutDerajat);
        this.taliBusur = hitungTaliBusurElips(super.semiMayorLuar,
super.semiMinorLuar, this.sudutDerajat);
        this.kelilingTembereng = panjangBusur + taliBusur;
        return this.kelilingTembereng;
    }

    public double hitungKelilingTembereng(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        if (sudutDerajat <= 0 || sudutDerajat >= 360) {
            throw new IllegalArgumentException("Sudut tembereng elips harus
lebih dari 0 dan kurang dari 360 derajat!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        this.kelilingElips = super.hitungKeliling(semiMayorLuar,
semiMinorLuar);
        this.theta = Math.toRadians(sudutDerajat);
        this.panjangBusur = hitungPanjangBusurElips(semiMayorLuar,
semiMinorLuar, sudutDerajat);
        this.taliBusur = hitungTaliBusurElips(semiMayorLuar, semiMinorLuar,
sudutDerajat);
        this.kelilingTembereng = panjangBusur + taliBusur;
        return this.kelilingTembereng;
    }

    private double hitungPanjangBusurElips(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        double theta = Math.toRadians(sudutDerajat);
        jumlahSegmen = 1000;
        h = theta / jumlahSegmen;
        total = integrandBusur(semiMayorLuar, semiMinorLuar, 0)
                + integrandBusur(semiMayorLuar, semiMinorLuar, theta);

        for (int i = 1; i < jumlahSegmen; i++) {
            double t = i * h;
            total += (i % 2 == 0 ? 2 : 4) * integrandBusur(semiMayorLuar,
semiMinorLuar, t);
        }
```

```
        panjangBusur = total * h / 3.0;
        return panjangBusur;
    }

    private double integrandBusur(double semiMayorLuar, double
semiMinorLuar, double t) {
        integrandBusur = Math.sqrt(Math.pow(-semiMayorLuar * Math.sin(t),
2) + Math.pow(semiMinorLuar * Math.cos(t), 2));
        return integrandBusur;
    }

    private double hitungTaliBusurElips(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        theta = Math.toRadians(sudutDerajat);
        taliBusur = Math.sqrt(
                Math.pow(semiMayorLuar * Math.cos(theta) - semiMayorLuar,
2)
                + Math.pow(semiMinorLuar * Math.sin(theta), 2)
        );
        return taliBusur;
    }

    @Override
    public void run() {
        if (this.isManual) {
            this.hitungLuasTembereng(semiMayorLuar, semiMinorLuar,
sudutDerajat);
            this.hitungKelilingTembereng(semiMayorLuar, semiMinorLuar,
sudutDerajat);
        } else {
            this.hitungLuas();
            this.hitungKeliling();
        }
    }
}
```

3.10 Class CincinElips3D.java

```
public class CincinElips3D extends BolaElips implements Runnable {

    public double radiusDalam, volumeDalam, luasPermukaanDalam,
volumeCincinElips3D, luasPermukaanCincinElips3D;
    public double luasElips, kelilingElips, volumeLuar, luasPermukaanLuar,
luasLingkaranDalam, kelilingLingkaranDalam;

    public CincinElips3D(double semiMayorLuar, double semiMinorLuar, double
sumbuC, double radiusDalam, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, sumbuC, isManual);
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
```

```
            throw new IllegalArgumentException("Nilai semi mayor dan semi
minor cincin elips 3D harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Nilai semi mayor harus
lebih besar atau sama dengan semi minor cincin elips 3D!");
        }
        if (radiusDalam <= 0) {
            throw new IllegalArgumentException("Radius dalam cincin elips
3D harus lebih besar dari 0!");
        }
        if (radiusDalam >= semiMayorLuar || radiusDalam >= semiMinorLuar) {
            throw new IllegalArgumentException("Radius dalam cincin elips
3D harus lebih kecil dari semi mayor dan semi minor luar!");
        }
        if (sumbuC <= 0) {
            throw new IllegalArgumentException("Nilai sumbu C bola elips
harus lebih besar dari 0!");
        }
        this.radiusDalam = radiusDalam;
    }

    public double hitungVolumeCincinElips3D() {
        this.luasElips = super.hitungLuas(super.semiMayorLuar,
super.semiMinorLuar);
        this.luasLingkaranDalam = PI * this.radiusDalam * this.radiusDalam;

        this.volumeLuar = this.luasElips * super.sumbuC;
        this.volumeDalam = luasLingkaranDalam * super.sumbuC;
        this.volumeCincinElips3D = this.volumeLuar - this.volumeDalam;
        return this.volumeCincinElips3D;
    }

    public double hitungVolumeCincinElips3D(double semiMayorLuar, double
semiMinorLuar, double sumbuC, double radiusDalam) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Nilai semi mayor dan semi
minor cincin elips 3D harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Nilai semi mayor harus
lebih besar atau sama dengan semi minor cincin elips 3D!");
        }
        if (radiusDalam <= 0) {
            throw new IllegalArgumentException("Radius dalam cincin elips
3D harus lebih besar dari 0!");
        }
        if (radiusDalam >= semiMayorLuar || radiusDalam >= semiMinorLuar) {
            throw new IllegalArgumentException("Radius dalam cincin elips
3D harus lebih kecil dari semi mayor dan semi minor luar!");
        }
        if (sumbuC <= 0) {
```

```
            throw new IllegalArgumentException("Nilai sumbu C bola elips
harus lebih besar dari 0!");
        }
        this.luasElips = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        this.luasLingkaranDalam = PI * radiusDalam * radiusDalam;

        this.volumeLuar = this.luasElips * sumbuC;
        this.volumeDalam = luasLingkaranDalam * sumbuC;
        this.volumeCincinElips3D = this.volumeLuar - this.volumeDalam;
        return this.volumeCincinElips3D;
    }

    public double hitungLuasPermukaanCincinElips3D() {
        this.luasElips = super.hitungLuas(super.semiMayorLuar,
super.semiMinorLuar);
        this.kelilingElips = super.hitungKeliling(super.semiMayorLuar,
super.semiMinorLuar);

        this.luasLingkaranDalam = PI * this.radiusDalam * this.radiusDalam;
        this.kelilingLingkaranDalam = 2 * PI * this.radiusDalam;

        this.luasPermukaanLuar = (2 * this.luasElips) + (this.kelilingElips
* super.sumbuC);
        this.luasPermukaanDalam = kelilingLingkaranDalam * super.sumbuC;
        this.luasPermukaanCincinElips3D = (2 * (this.luasElips -
luasLingkaranDalam))
                + (this.kelilingElips * super.sumbuC)
                + this.luasPermukaanDalam;
        return this.luasPermukaanCincinElips3D;
    }

    public double hitungLuasPermukaanCincinElips3D(double semiMayorLuar,
double semiMinorLuar, double sumbuC, double radiusDalam) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Nilai semi mayor dan semi
minor cincin elips 3D harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Nilai semi mayor harus
lebih besar atau sama dengan semi minor cincin elips 3D!");
        }
        if (radiusDalam <= 0) {
            throw new IllegalArgumentException("Radius dalam cincin elips
3D harus lebih besar dari 0!");
        }
        if (radiusDalam >= semiMayorLuar || radiusDalam >= semiMinorLuar) {
            throw new IllegalArgumentException("Radius dalam cincin elips
3D harus lebih kecil dari semi mayor dan semi minor luar!");
        }
        if (sumbuC <= 0) {
            throw new IllegalArgumentException("Nilai sumbu C bola elips
harus lebih besar dari 0!");
```

```
        }
        this.luasElips = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        this.kelilingElips = super.hitungKeliling(semiMayorLuar,
semiMinorLuar);

        this.luasLingkaranDalam = PI * radiusDalam * radiusDalam;
        this.kelilingLingkaranDalam = 2 * PI * radiusDalam;

        this.luasPermukaanLuar = (2 * this.luasElips) + (this.kelilingElips
* sumbuC);
        this.luasPermukaanDalam = this.kelilingLingkaranDalam * sumbuC;
        this.luasPermukaanCincinElips3D = (2 * (this.luasElips -
this.luasLingkaranDalam))
                + (this.kelilingElips * sumbuC)
                + this.luasPermukaanDalam;
        return this.luasPermukaanCincinElips3D;
    }

    @Override
    public void run() {
        if (this.isManual) {
            this.hitungLuasPermukaanCincinElips3D(semiMayorLuar,
semiMinorLuar, sumbuC, radiusDalam);
            this.hitungVolumeCincinElips3D(semiMayorLuar, semiMinorLuar,
sumbuC, radiusDalam);
        } else {
            this.hitungLuasPermukaanCincinElips3D();
            this.hitungVolumeCincinElips3D();
        }
    }
}
```

3.11 Class JuringElips3D.java

```
public class JuringElips3D extends BolaElips implements Runnable {

    public double sudutDerajat, tinggiJuring, luasAlasJuring,
kelilingAlasJuring, volumeJuringElips3D, luasPermukaanJuringElips3D;
    public double luasElips, panjangBusur, rataRataJariJari, kelilingElips;
    public double sisiAwal, sisiAkhir, theta, integrandBusur,
jarakPusatKeTitikElips, h, total, jumlahSegmen;

    public JuringElips3D(double semiMayorLuar, double semiMinorLuar, double
tinggiJuring, double sudutDerajat, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, tinggiJuring, isManual);
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
```

```
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        if (tinggiJuring <= 0) {
            throw new IllegalArgumentException("Tinggi juring elips 3D
harus lebih besar dari 0!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips 3D harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        this.tinggiJuring = tinggiJuring;
        this.sudutDerajat = sudutDerajat;
    }

    public double hitungVolumeJuringElips3D() {
        this.luasAlasJuring = (this.sudutDerajat / 360.0) * super.luas;
        this.volumeJuringElips3D = this.luasAlasJuring * this.tinggiJuring;
        return this.volumeJuringElips3D;
    }

    public double hitungVolumeJuringElips3D(double semiMayorLuar, double
semiMinorLuar, double tinggiJuring, double sudutDerajat) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        if (tinggiJuring <= 0) {
            throw new IllegalArgumentException("Tinggi juring elips 3D
harus lebih besar dari 0!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips 3D harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        luasElips = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        this.luasAlasJuring = (sudutDerajat / 360.0) * luasElips;
        this.volumeJuringElips3D = this.luasAlasJuring * tinggiJuring;
        return this.volumeJuringElips3D;
    }

    public double hitungLuasPermukaanJuringElips3D() {
        panjangBusur = hitungPanjangBusurElips(super.semiMayorLuar,
super.semiMinorLuar, this.sudutDerajat);
        if (Math.abs(this.sudutDerajat - 360.0) < 0.0000001) {
            sisiAwal = 0;
            sisiAkhir = 0;
        } else {
```

```
            sisiAwal = super.semiMayorLuar;
            sisiAkhir = hitungJarakPusatKeTitikElips(super.semiMayorLuar,
super.semiMinorLuar, this.sudutDerajat);
        }
        rataRataJariJari = (sisiAwal + sisiAkhir) / 2.0;
        this.kelilingAlasJuring = panjangBusur + sisiAwal + sisiAkhir;
        this.luasAlasJuring = (this.sudutDerajat / 360.0) * super.luas;
        this.luasPermukaanJuringElips3D = (2 * this.luasAlasJuring) +
(this.kelilingAlasJuring * this.tinggiJuring);
        return this.luasPermukaanJuringElips3D;
    }

    public double hitungLuasPermukaanJuringElips3D(double semiMayorLuar,
double semiMinorLuar, double tinggiJuring, double sudutDerajat) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        if (tinggiJuring <= 0) {
            throw new IllegalArgumentException("Tinggi juring elips 3D
harus lebih besar dari 0!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips 3D harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        luasElips = super.hitungLuas(semiMayorLuar, semiMinorLuar);
        kelilingElips = super.hitungKeliling(semiMayorLuar, semiMinorLuar);
        panjangBusur = hitungPanjangBusurElips(semiMayorLuar,
semiMinorLuar, sudutDerajat);
        if (Math.abs(sudutDerajat - 360.0) < 0.0000001) {
            sisiAwal = 0;
            sisiAkhir = 0;
        } else {
            sisiAwal = semiMayorLuar;
            sisiAkhir = hitungJarakPusatKeTitikElips(semiMayorLuar,
semiMinorLuar, sudutDerajat);
        }
        rataRataJariJari = (sisiAwal + sisiAkhir) / 2.0;
        this.kelilingAlasJuring = panjangBusur + sisiAwal + sisiAkhir;
        this.luasAlasJuring = (sudutDerajat / 360.0) * luasElips;
        this.luasPermukaanJuringElips3D = (2 * this.luasAlasJuring) +
(this.kelilingAlasJuring * tinggiJuring);
        return this.luasPermukaanJuringElips3D;
    }

    public double hitungPanjangBusurElips(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
```

```
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        if (tinggiJuring <= 0) {
            throw new IllegalArgumentException("Tinggi juring elips 3D
harus lebih besar dari 0!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips 3D harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        theta = Math.toRadians(sudutDerajat);
        jumlahSegmen = 1000;
        h = theta / jumlahSegmen;
        total = integrandBusur(semiMayorLuar, semiMinorLuar, 0)
                + integrandBusur(semiMayorLuar, semiMinorLuar, theta);

        for (int i = 1; i < jumlahSegmen; i++) {
            double t = i * h;
            total += (i % 2 == 0 ? 2 : 4) * integrandBusur(semiMayorLuar,
semiMinorLuar, t);
        }

        panjangBusur = total * h / 3.0;
        return panjangBusur;
    }

    public double integrandBusur(double semiMayorLuar, double
semiMinorLuar, double t) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        if (tinggiJuring <= 0) {
            throw new IllegalArgumentException("Tinggi juring elips 3D
harus lebih besar dari 0!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips 3D harus
lebih dari 0 dan maksimal 360 derajat!");
        }

        integrandBusur = Math.sqrt(Math.pow(-semiMayorLuar * Math.sin(t),
```

```
2) + Math.pow(semiMinorLuar * Math.cos(t), 2));
        return integrandBusur;
    }

    public double hitungJarakPusatKeTitikElips(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        if (tinggiJuring <= 0) {
            throw new IllegalArgumentException("Tinggi juring elips 3D
harus lebih besar dari 0!");
        }
        if (sudutDerajat <= 0 || sudutDerajat > 360) {
            throw new IllegalArgumentException("Sudut juring elips 3D harus
lebih dari 0 dan maksimal 360 derajat!");
        }
        theta = Math.toRadians(sudutDerajat);

        jarakPusatKeTitikElips = Math.sqrt(
                Math.pow(semiMayorLuar * Math.cos(theta), 2)
                + Math.pow(semiMinorLuar * Math.sin(theta), 2)
        );
        return jarakPusatKeTitikElips;
    }

    @Override
    public void run() {
        super.run();
        if (this.isManual) {
            this.hitungLuasPermukaanJuringElips3D(semiMayorLuar,
semiMinorLuar, tinggiJuring, sudutDerajat);
            this.hitungVolumeJuringElips3D(semiMayorLuar, semiMinorLuar,
tinggiJuring, sudutDerajat);
        } else {
            this.hitungLuasPermukaanJuringElips3D();
            this.hitungVolumeJuringElips3D();
        }
    }
}
```

3.12 Class TemberengElips3D.java

```
public class TemberengElips3D extends BolaElips implements Runnable {

    public double sudutDerajat, tinggiTembereng, luasAlasTembereng,
kelilingAlasTembereng, volumeTemberengElips3D,
luasPermukaanTemberengElips3D;
    public double theta, panjangBusur, taliBusur, kelilingElips,
integrandBusur, jumlahSegmen, h, total;

    public TemberengElips3D(double semiMayorLuar, double semiMinorLuar,
double tinggiTembereng, double sudutDerajat, boolean isManual) {
        super(semiMayorLuar, semiMinorLuar, tinggiTembereng, isManual);
        if (tinggiTembereng <= 0) {
            throw new IllegalArgumentException("Tinggi tembereng elips 3D
harus lebih besar dari 0!");
        }
        if (sudutDerajat <= 0 || sudutDerajat >= 360) {
            throw new IllegalArgumentException("Sudut tembereng elips 3D
harus lebih dari 0 dan kurang dari 360 derajat!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }

        this.tinggiTembereng = tinggiTembereng;
        this.sudutDerajat = sudutDerajat;
    }

    public double hitungVolumeTemberengElips3D() {
        this.theta = Math.toRadians(this.sudutDerajat);
        this.luasAlasTembereng = 0.5 * super.semiMayorLuar *
super.semiMinorLuar * (theta - Math.sin(theta));
        this.volumeTemberengElips3D = this.luasAlasTembereng *
this.tinggiTembereng;
        return this.volumeTemberengElips3D;
    }

    public double hitungVolumeTemberengElips3D(double semiMayorLuar, double
semiMinorLuar, double tinggiTembereng, double sudutDerajat) {
        if (tinggiTembereng <= 0) {
            throw new IllegalArgumentException("Tinggi tembereng elips 3D
harus lebih besar dari 0!");
        }
        if (sudutDerajat <= 0 || sudutDerajat >= 360) {
            throw new IllegalArgumentException("Sudut tembereng elips 3D
```

```
harus lebih dari 0 dan kurang dari 360 derajat!");
        }
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        this.theta = Math.toRadians(sudutDerajat);
        this.luasAlasTembereng = 0.5 * semiMayorLuar * semiMinorLuar *
(theta - Math.sin(theta));
        this.volumeTemberengElips3D = this.luasAlasTembereng *
tinggiTembereng;
        return this.volumeTemberengElips3D;
    }

    public double hitungLuasPermukaanTemberengElips3D() {
        this.theta = Math.toRadians(this.sudutDerajat);
        this.panjangBusur = hitungPanjangBusurElips(super.semiMayorLuar,
super.semiMinorLuar, this.sudutDerajat);
        this.taliBusur = hitungTaliBusurElips(super.semiMayorLuar,
super.semiMinorLuar, this.sudutDerajat);
        this.kelilingAlasTembereng = panjangBusur + taliBusur;
        this.luasAlasTembereng = 0.5 * super.semiMayorLuar *
super.semiMinorLuar * (theta - Math.sin(theta));
        this.luasPermukaanTemberengElips3D = (2 * this.luasAlasTembereng) +
(this.kelilingAlasTembereng * this.tinggiTembereng);
        return this.luasPermukaanTemberengElips3D;
    }

    public double hitungLuasPermukaanTemberengElips3D(double semiMayorLuar,
double semiMinorLuar, double tinggiTembereng, double sudutDerajat) {
        if (semiMayorLuar <= 0 || semiMinorLuar <= 0) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
dan semi minor harus lebih besar dari 0!");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new IllegalArgumentException("Panjang sumbu semi mayor
harus lebih besar atau sama dengan sumbu semi minor!");
        }
        if (sudutDerajat <= 0 || sudutDerajat >= 360) {
            throw new IllegalArgumentException("Sudut tembereng elips 3D
harus lebih dari 0 dan kurang dari 360 derajat!");
        }
        if (tinggiTembereng <= 0) {
            throw new IllegalArgumentException("Tinggi tembereng elips 3D
harus lebih besar dari 0!");
        }

        this.kelilingElips = super.hitungKeliling(semiMayorLuar,
```

```
semiMinorLuar);
        this.theta = Math.toRadians(sudutDerajat);
        this.panjangBusur = hitungPanjangBusurElips(semiMayorLuar,
semiMinorLuar, sudutDerajat);
        this.taliBusur = hitungTaliBusurElips(semiMayorLuar, semiMinorLuar,
sudutDerajat);
        this.kelilingAlasTembereng = panjangBusur + taliBusur;
        this.luasAlasTembereng = 0.5 * semiMayorLuar * semiMinorLuar *
(theta - Math.sin(theta));
        this.luasPermukaanTemberengElips3D = (2 * this.luasAlasTembereng) +
(this.kelilingAlasTembereng * tinggiTembereng);
        return this.luasPermukaanTemberengElips3D;
    }

    private double hitungPanjangBusurElips(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        theta = Math.toRadians(sudutDerajat);
        jumlahSegmen = 1000;
        h = theta / jumlahSegmen;
        total = integrandBusur(semiMayorLuar, semiMinorLuar, 0)
                + integrandBusur(semiMayorLuar, semiMinorLuar, theta);

        for (int i = 1; i < jumlahSegmen; i++) {
            double t = i * h;
            total += (i % 2 == 0 ? 2 : 4) * integrandBusur(semiMayorLuar,
semiMinorLuar, t);
        }

        panjangBusur = total * h / 3.0;
        return panjangBusur;
    }

    private double integrandBusur(double semiMayorLuar, double
semiMinorLuar, double t) {
        integrandBusur = Math.sqrt(
                Math.pow(semiMayorLuar * Math.sin(t), 2)
                + Math.pow(semiMinorLuar * Math.cos(t), 2)
        );
        return integrandBusur;
    }

    private double hitungTaliBusurElips(double semiMayorLuar, double
semiMinorLuar, double sudutDerajat) {
        theta = Math.toRadians(sudutDerajat);
        taliBusur = Math.sqrt(
                Math.pow(semiMayorLuar * Math.cos(theta) - semiMayorLuar,
2)
                + Math.pow(semiMinorLuar * Math.sin(theta), 2)
        );
        return taliBusur;
    }
```

```
    @Override
    public void run() {
        super.run();
        if (this.isManual) {
            this.hitungLuasPermukaanTemberengElips3D(semiMayorLuar,
semiMinorLuar, tinggiTembereng, sudutDerajat);
            this.hitungVolumeTemberengElips3D(semiMayorLuar, semiMinorLuar,
tinggiTembereng, sudutDerajat);
        } else {
            this.hitungLuasPermukaanTemberengElips3D();
            this.hitungVolumeTemberengElips3D();
        }
    }
}
```

3.13 Class GeometriGUI.java

```
import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.Random;

public class GeometriGUI extends JFrame {

    public static class InvalidInputException extends Exception {
        public InvalidInputException(String message) {
            super(message);
        }
    }

    public static class ThreadComputationException extends RuntimeException
{
        public ThreadComputationException(String message, Throwable cause)
{
            super(message, cause);
        }
    }

    public JButton btnGenerate;
    public JComboBox<String> cbMode;
    public JTextField txtJumlahData, txtSemiMayorLuar, txtSemiMinorLuar,
txtTinggi, txtSudut;
    public JTextField txtSemiMayorDalam, txtSemiMinorDalam, txtRadiusDalam,
txtFaktorAtas;

    public JTable tabelElips, tabelJuring2D, tabelCincin2D,
tabelTembereng2D;
    public JTable tabelBolaElips, tabelLimasElips, tabelLimasTerpancung,
```

```
tabelPrismaElips;
    public JTable tabelJuring3D, tabelCincin3D, tabelTembereng3D;

    public DefaultTableModel modelElips, modelJuring2D, modelCincin2D,
modelTembereng2D;
    public DefaultTableModel modelBolaElips, modelLimasElips,
modelLimasTerpancung, modelPrismaElips;
    public DefaultTableModel modelJuring3D, modelCincin3D,
modelTembereng3D;

    public JProgressBar pbElips, pbJuring2D, pbCincin2D, pbTembereng2D;
    public JProgressBar pbBolaElips, pbLimasElips, pbLimasTerpancung,
pbPrismaElips;
    public JProgressBar pbJuring3D, pbCincin3D, pbTembereng3D;

    public GeometriGUI() {
        super("Sistem Komputasi Geometri Elips Multithreading");

        btnGenerate = new JButton("Generate Data");
        cbMode = new JComboBox<>(new String[]{"Opsi 1: Generate Angka
Random", "Opsi 2: Gunakan Input Manual"});

        txtJumlahData = new JTextField("1000");
        txtSemiMayorLuar = new JTextField("10");
        txtSemiMinorLuar = new JTextField("6");
        txtTinggi = new JTextField("12");
        txtSudut = new JTextField("90");
        txtSemiMayorDalam = new JTextField("4");
        txtSemiMinorDalam = new JTextField("2");
        txtRadiusDalam = new JTextField("2");
        txtFaktorAtas = new JTextField("0.6");

        modelElips = new DefaultTableModel(new String[]{"No", "Semi Mayor
Luar", "Semi Minor Luar", "Luas Elips", "Keliling Elips"}, 0);
        modelJuring2D = new DefaultTableModel(new String[]{"No", "Semi
Mayor Luar", "Semi Minor Luar", "Sudut", "Luas Juring", "Keliling Juring"},
0);
        modelCincin2D = new DefaultTableModel(new String[]{"No", "Semi
Mayor Luar", "Semi Minor Luar", "Semi Mayor Dalam", "Semi Minor Dalam",
"Luas Cincin", "Keliling Cincin"}, 0);
        modelTembereng2D = new DefaultTableModel(new String[]{"No", "Semi
Mayor Luar", "Semi Minor Luar", "Sudut", "Luas Tembereng", "Keliling
Tembereng"}, 0);

        modelBolaElips = new DefaultTableModel(new String[]{"No", "Semi
Mayor Luar", "Semi Minor Luar", "Sumbu C", "Volume", "Luas Permukaan"}, 0);
        modelLimasElips = new DefaultTableModel(new String[]{"No", "Semi
Mayor Luar", "Semi Minor Luar", "Tinggi", "Volume", "Luas Permukaan"}, 0);
        modelLimasTerpancung = new DefaultTableModel(new String[]{"No",
"Semi Mayor Luar", "Semi Minor Luar", "Tinggi", "Faktor Atas", "Volume",
"Luas Permukaan"}, 0);
        modelPrismaElips = new DefaultTableModel(new String[]{"No", "Semi
```

```
Mayor Luar", "Semi Minor Luar", "Tinggi", "Volume", "Luas Permukaan"}, 0);

        modelJuring3D = new DefaultTableModel(new String[]{"No", "Semi
Mayor Luar", "Semi Minor Luar", "Tinggi", "Sudut", "Volume", "Luas
Permukaan"}, 0);
        modelCincin3D = new DefaultTableModel(new String[]{"No", "Semi
Mayor Luar", "Semi Minor Luar", "Tinggi", "Radius Dalam", "Volume", "Luas
Permukaan"}, 0);
        modelTembereng3D = new DefaultTableModel(new String[]{"No", "Semi
Mayor Luar", "Semi Minor Luar", "Tinggi", "Sudut", "Volume", "Luas
Permukaan"}, 0);

        tabelElips = new JTable(modelElips);
        tabelJuring2D = new JTable(modelJuring2D);
        tabelCincin2D = new JTable(modelCincin2D);
        tabelTembereng2D = new JTable(modelTembereng2D);
        tabelBolaElips = new JTable(modelBolaElips);
        tabelLimasElips = new JTable(modelLimasElips);
        tabelLimasTerpancung = new JTable(modelLimasTerpancung);
        tabelPrismaElips = new JTable(modelPrismaElips);
        tabelJuring3D = new JTable(modelJuring3D);
        tabelCincin3D = new JTable(modelCincin3D);
        tabelTembereng3D = new JTable(modelTembereng3D);

        pbElips = progressBar();
        pbJuring2D = progressBar();
        pbCincin2D = progressBar();
        pbTembereng2D = progressBar();
        pbBolaElips = progressBar();
        pbLimasElips = progressBar();
        pbLimasTerpancung = progressBar();
        pbPrismaElips = progressBar();
        pbJuring3D = progressBar();
        pbCincin3D = progressBar();
        pbTembereng3D = progressBar();

        Container container = getContentPane();
        container.setLayout(new BorderLayout(8, 8));

        JPanel panelInput = new JPanel(new GridLayout(6, 4, 6, 6));
        panelInput.setBorder(BorderFactory.createTitledBorder("Input
Data"));
        panelInput.add(new JLabel("Pilih Opsi:"));
        panelInput.add(cbMode);
        panelInput.add(new JLabel("Jumlah Data:"));
        panelInput.add(txtJumlahData);

        panelInput.add(new JLabel("Semi Mayor Luar:"));
        panelInput.add(txtSemiMayorLuar);
        panelInput.add(new JLabel("Semi Minor Luar:"));
        panelInput.add(txtSemiMinorLuar);
```

```
        panelInput.add(new JLabel("Tinggi / Sumbu C:"));
        panelInput.add(txtTinggi);
        panelInput.add(new JLabel("Sudut Juring/Tembereng:"));
        panelInput.add(txtSudut);

        panelInput.add(new JLabel("Semi Mayor Dalam:"));
        panelInput.add(txtSemiMayorDalam);
        panelInput.add(new JLabel("Semi Minor Dalam:"));
        panelInput.add(txtSemiMinorDalam);

        panelInput.add(new JLabel("Radius Dalam 3D:"));
        panelInput.add(txtRadiusDalam);
        panelInput.add(new JLabel("Faktor Atas Terpancung:"));
        panelInput.add(txtFaktorAtas);

        panelInput.add(new JLabel(""));
        panelInput.add(btnGenerate);
        panelInput.add(new JLabel(""));
        panelInput.add(new JLabel(""));

        container.add(panelInput, BorderLayout.NORTH);

        JTabbedPane tabbedPane = new JTabbedPane();
        tabbedPane.add("Elips", scroll(tabelElips, "Tabel Hasil Elips"));
        tabbedPane.add("Juring 2D", scroll(tabelJuring2D, "Tabel Hasil
Juring Elips 2D"));
        tabbedPane.add("Cincin 2D", scroll(tabelCincin2D, "Tabel Hasil
Cincin Elips 2D"));
        tabbedPane.add("Tembereng 2D", scroll(tabelTembereng2D, "Tabel
Hasil Tembereng Elips 2D"));
        tabbedPane.add("Bola Elips", scroll(tabelBolaElips, "Tabel Hasil
Bola Elips 3D"));
        tabbedPane.add("Limas Elips", scroll(tabelLimasElips, "Tabel Hasil
Kerucut/Limas Alas Elips"));
        tabbedPane.add("Terpancung", scroll(tabelLimasTerpancung, "Tabel
Hasil Kerucut/Limas Elips Terpancung"));
        tabbedPane.add("Prisma Elips", scroll(tabelPrismaElips, "Tabel
Hasil Tabung/Prisma Elips"));
        tabbedPane.add("Juring 3D", scroll(tabelJuring3D, "Tabel Hasil
Juring Elips 3D"));
        tabbedPane.add("Cincin 3D", scroll(tabelCincin3D, "Tabel Hasil
Cincin Elips 3D"));
        tabbedPane.add("Tembereng 3D", scroll(tabelTembereng3D, "Tabel
Hasil Tembereng Elips 3D"));
        container.add(tabbedPane, BorderLayout.CENTER);

        JPanel panelProgress = new JPanel(new GridLayout(11, 2, 5, 3));
        panelProgress.setBorder(BorderFactory.createTitledBorder("Progress
Thread"));
        addProgress(panelProgress, "Thread 1 - Elips", pbElips);
        addProgress(panelProgress, "Thread 2 - Juring 2D", pbJuring2D);
        addProgress(panelProgress, "Thread 3 - Cincin 2D", pbCincin2D);
```

```
        addProgress(panelProgress, "Thread 4 - Tembereng 2D",
pbTembereng2D);
        addProgress(panelProgress, "Thread 5 - Bola Elips", pbBolaElips);
        addProgress(panelProgress, "Thread 6 - Limas Elips", pbLimasElips);
        addProgress(panelProgress, "Thread 7 - Terpancung",
pbLimasTerpancung);
        addProgress(panelProgress, "Thread 8 - Prisma Elips",
pbPrismaElips);
        addProgress(panelProgress, "Thread 9 - Juring 3D", pbJuring3D);
        addProgress(panelProgress, "Thread 10 - Cincin 3D", pbCincin3D);
        addProgress(panelProgress, "Thread 11 - Tembereng 3D",
pbTembereng3D);
        container.add(panelProgress, BorderLayout.SOUTH);

        btnGenerate.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                jalankanMasterThread();
            }
        });

        setSize(1250, 820);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }

    private JProgressBar progressBar() {
        JProgressBar pb = new JProgressBar(0, 1000);
        pb.setStringPainted(true);
        return pb;
    }

    private JScrollPane scroll(JTable tabel, String judul) {
        JScrollPane sp = new JScrollPane(tabel);
        sp.setBorder(BorderFactory.createTitledBorder(judul));
        return sp;
    }

    private void addProgress(JPanel panel, String label, JProgressBar pb) {
        panel.add(new JLabel(" " + label + ":"));
        panel.add(pb);
    }

    public int validasiJumlahData() throws InvalidInputException {
        String raw = txtJumlahData.getText().trim();
        if (raw.isEmpty()) {
            throw new InvalidInputException("Field Jumlah Data tidak boleh
kosong.");
        }
        int jumlah;
        try {
            jumlah = Integer.parseInt(raw);
```

```
        } catch (NumberFormatException e) {
            throw new InvalidInputException("Jumlah Data harus berupa angka
bulat. Contoh: 1000");
        }
        if (jumlah <= 0) {
            throw new InvalidInputException("Jumlah Data harus lebih dari
0.");
        }
        if (jumlah > 100000) {
            throw new InvalidInputException("Jumlah Data terlalu besar.
Maksimal 100000.");
        }
        return jumlah;
    }

    public double[] validasiInputManual() throws InvalidInputException {
        double semiMayorLuar =
parsePositive(txtSemiMayorLuar.getText().trim(), "Semi Mayor Luar");
        double semiMinorLuar =
parsePositive(txtSemiMinorLuar.getText().trim(), "Semi Minor Luar");
        double tinggi = parsePositive(txtTinggi.getText().trim(), "Tinggi /
Sumbu C");
        double sudut = parseRange(txtSudut.getText().trim(), "Sudut", 0,
360, false, true);
        double semiMayorDalam =
parsePositive(txtSemiMayorDalam.getText().trim(), "Semi Mayor Dalam");
        double semiMinorDalam =
parsePositive(txtSemiMinorDalam.getText().trim(), "Semi Minor Dalam");
        double radiusDalam = parsePositive(txtRadiusDalam.getText().trim(),
"Radius Dalam 3D");
        double faktorAtas = parseRange(txtFaktorAtas.getText().trim(),
"Faktor Atas Terpancung", 0, 1, false, false);

        if (semiMayorLuar > 1000000 || semiMinorLuar > 1000000 || tinggi >
1000000) {
            throw new InvalidInputException("Nilai sumbu/tinggi terlalu
besar. Maksimal 1000000.");
        }
        if (semiMayorLuar < semiMinorLuar) {
            throw new InvalidInputException("Semi Mayor Luar tidak boleh
lebih kecil dari Semi Minor Luar.");
        }
        if (semiMayorDalam < semiMinorDalam) {
            throw new InvalidInputException("Semi Mayor Dalam tidak boleh
lebih kecil dari Semi Minor Dalam.");
        }
        if (semiMayorDalam >= semiMayorLuar || semiMinorDalam >=
semiMinorLuar) {
            throw new InvalidInputException("Semi mayor/minor dalam harus
lebih kecil dari semi mayor/minor luar.");
        }
        if (radiusDalam >= semiMayorLuar || radiusDalam >= semiMinorLuar) {
```

```
            throw new InvalidInputException("Radius Dalam 3D harus lebih
kecil dari Semi Mayor Luar dan Semi Minor Luar.");
        }
        return new double[]{semiMayorLuar, semiMinorLuar, tinggi, sudut,
semiMayorDalam, semiMinorDalam, radiusDalam, faktorAtas};
    }

    private double parsePositive(String raw, String namaField) throws
InvalidInputException {
        if (raw.isEmpty()) {
            throw new InvalidInputException("Field " + namaField + " tidak
boleh kosong.");
        }
        double value;
        try {
            value = Double.parseDouble(raw);
        } catch (NumberFormatException e) {
            throw new InvalidInputException(namaField + " harus berupa
angka desimal. Contoh: 10.5");
        }
        if (value <= 0) {
            throw new InvalidInputException(namaField + " harus lebih dari
0.");
        }
        return value;
    }

    private double parseRange(String raw, String namaField, double min,
double max, boolean includeMin, boolean includeMax) throws
InvalidInputException {
        if (raw.isEmpty()) {
            throw new InvalidInputException("Field " + namaField + " tidak
boleh kosong.");
        }
        double value;
        try {
            value = Double.parseDouble(raw);
        } catch (NumberFormatException e) {
            throw new InvalidInputException(namaField + " harus berupa
angka desimal.");
        }

        boolean validMin = includeMin ? value >= min : value > min;
        boolean validMax = includeMax ? value <= max : value < max;
        if (!validMin || !validMax) {
            String minText = includeMin ? ">= " + min : "> " + min;
            String maxText = includeMax ? "<= " + max : "< " + max;
            throw new InvalidInputException(namaField + " harus " + minText
+ " dan " + maxText + ".");
        }
        return value;
    }
```

```
    public void jalankanMasterThread() {
        btnGenerate.setEnabled(false);
        resetTabel();

        Thread masterThread = new Thread(() -> {
            final int jumlahData;
            final boolean isManual = cbMode.getSelectedIndex() == 1;
            final double manualSemiMayorLuar, manualSemiMinorLuar,
manualTinggi, manualSudut;
            final double manualSemiMayorDalam, manualSemiMinorDalam,
manualRadiusDalam, manualFaktorAtas;

            try {
                jumlahData = validasiJumlahData();
                if (isManual) {
                    double[] input = validasiInputManual();
                    manualSemiMayorLuar = input[0];
                    manualSemiMinorLuar = input[1];
                    manualTinggi = input[2];
                    manualSudut = input[3];
                    manualSemiMayorDalam = input[4];
                    manualSemiMinorDalam = input[5];
                    manualRadiusDalam = input[6];
                    manualFaktorAtas = input[7];
                } else {
                    manualSemiMayorLuar = 10;
                    manualSemiMinorLuar = 6;
                    manualTinggi = 12;
                    manualSudut = 90;
                    manualSemiMayorDalam = 4;
                    manualSemiMinorDalam = 2;
                    manualRadiusDalam = 2;
                    manualFaktorAtas = 0.6;
                }
            } catch (InvalidInputException e) {
                SwingUtilities.invokeLater(() -> {
                    JOptionPane.showMessageDialog(GeometriGUI.this,
                            "Input Tidak Valid:\n" + e.getMessage(),
                            "Error Validasi",
                            JOptionPane.ERROR_MESSAGE);
                    btnGenerate.setEnabled(true);
                });
                return;
            }

            SwingUtilities.invokeLater(() ->
setMaksimumProgress(jumlahData));

            double[] dataA = new double[jumlahData];
            double[] dataB = new double[jumlahData];
            double[] dataTinggi = new double[jumlahData];
```

```
            double[] dataSudut = new double[jumlahData];
            double[] dataSemiMayorDalam = new double[jumlahData];
            double[] dataSemiMinorDalam = new double[jumlahData];
            double[] dataRadiusDalam = new double[jumlahData];
            double[] dataFaktorAtas = new double[jumlahData];

            Random rand = new Random();
            for (int i = 0; i < jumlahData; i++) {
                if (isManual) {
                    dataA[i] = manualSemiMayorLuar;
                    dataB[i] = manualSemiMinorLuar;
                    dataTinggi[i] = manualTinggi;
                    dataSudut[i] = manualSudut;
                    dataSemiMayorDalam[i] = manualSemiMayorDalam;
                    dataSemiMinorDalam[i] = manualSemiMinorDalam;
                    dataRadiusDalam[i] = manualRadiusDalam;
                    dataFaktorAtas[i] = manualFaktorAtas;
                } else {
                    dataA[i] = 1 + (rand.nextDouble() * 49);
                    dataB[i] = 1 + (rand.nextDouble() * 49);
                    if (dataA[i] < dataB[i]) {
                        double temp = dataA[i];
                        dataA[i] = dataB[i];
                        dataB[i] = temp;
                    }
                    dataTinggi[i] = 1 + (rand.nextDouble() * 49);
                    dataSudut[i] = 10 + (rand.nextDouble() * 340);
                    double skalaDalamAcak = 0.1 + (rand.nextDouble() *
0.8);
                    dataSemiMayorDalam[i] = dataA[i] * skalaDalamAcak;
                    dataSemiMinorDalam[i] = dataB[i] * skalaDalamAcak;
                    dataRadiusDalam[i] = dataB[i] * (0.1 +
(rand.nextDouble() * 0.7));
                    dataFaktorAtas[i] = 0.1 + (rand.nextDouble() * 0.8);
                }
            }

            Object[][] hasilElips = new Object[jumlahData][5];
            Object[][] hasilJuring2D = new Object[jumlahData][6];
            Object[][] hasilCincin2D = new Object[jumlahData][7];
            Object[][] hasilTembereng2D = new Object[jumlahData][6];
            Object[][] hasilBola = new Object[jumlahData][6];
            Object[][] hasilLimas = new Object[jumlahData][6];
            Object[][] hasilTerpancung = new Object[jumlahData][7];
            Object[][] hasilPrisma = new Object[jumlahData][6];
            Object[][] hasilJuring3D = new Object[jumlahData][7];
            Object[][] hasilCincin3D = new Object[jumlahData][7];
            Object[][] hasilTembereng3D = new Object[jumlahData][7];

            final Throwable[] errors = new Throwable[11];

            Thread t1 = buatThread("Elips", 0, pbElips, hasilElips,
```

```
jumlahData, dataA, dataB, dataTinggi, dataSudut, dataSemiMayorDalam,
dataSemiMinorDalam, dataRadiusDalam, dataFaktorAtas, isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips1 = new Elips(a, b, manual);
                        elips1 = jalankanPolymorphism(elips1);
                        return new Object[]{i + 1, f(a), f(b),
f(elips1.luas), f(elips1.keliling)};
                    }, errors);

            Thread t2 = buatThread("Juring Elips 2D", 1, pbJuring2D,
hasilJuring2D, jumlahData, dataA, dataB, dataTinggi, dataSudut,
dataSemiMayorDalam, dataSemiMinorDalam, dataRadiusDalam, dataFaktorAtas,
isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips2 = new JuringElips(a, b, sudut,
manual);
                        elips2 = jalankanPolymorphism(elips2);
                        JuringElips j = (JuringElips) elips2;
                        return new Object[]{i + 1, f(a), f(b), f(sudut),
f(j.luasJuring), f(j.kelilingJuring)};
                    }, errors);

            Thread t3 = buatThread("Cincin Elips 2D", 2, pbCincin2D,
hasilCincin2D, jumlahData, dataA, dataB, dataTinggi, dataSudut,
dataSemiMayorDalam, dataSemiMinorDalam, dataRadiusDalam, dataFaktorAtas,
isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips3 = new CincinElips(a, b,
semiMayorDalam, semiMinorDalam, manual);
                        elips3 = jalankanPolymorphism(elips3);
                        CincinElips c = (CincinElips) elips3;
                        return new Object[]{i + 1, f(a), f(b),
f(semiMayorDalam), f(semiMinorDalam), f(c.luasCincin),
f(c.kelilingCincin)};
                    }, errors);

            Thread t4 = buatThread("Tembereng Elips 2D", 3, pbTembereng2D,
hasilTembereng2D, jumlahData, dataA, dataB, dataTinggi, dataSudut,
dataSemiMayorDalam, dataSemiMinorDalam, dataRadiusDalam, dataFaktorAtas,
isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips4 = new TemberengElips(a, b, sudut,
manual);
                        elips4 = jalankanPolymorphism(elips4);
                        TemberengElips t = (TemberengElips) elips4;
                        return new Object[]{i + 1, f(a), f(b), f(sudut),
f(t.luasTembereng), f(t.kelilingTembereng)};
                    }, errors);
```

```
            Thread t5 = buatThread("Bola Elips", 4, pbBolaElips, hasilBola,
jumlahData, dataA, dataB, dataTinggi, dataSudut, dataSemiMayorDalam,
dataSemiMinorDalam, dataRadiusDalam, dataFaktorAtas, isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips5 = new BolaElips(a, b, tinggi, manual);
                        elips5 = jalankanPolymorphism(elips5);
                        BolaElips bola = (BolaElips) elips5;
                        return new Object[]{i + 1, f(a), f(b), f(tinggi),
f(bola.volumeBolaElips), f(bola.luasPermukaanBolaElips)};
                    }, errors);

            Thread t6 = buatThread("Limas Elips", 5, pbLimasElips,
hasilLimas, jumlahData, dataA, dataB, dataTinggi, dataSudut,
dataSemiMayorDalam, dataSemiMinorDalam, dataRadiusDalam, dataFaktorAtas,
isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips6 = new LimasElips(a, b, tinggi,
manual);
                        elips6 = jalankanPolymorphism(elips6);
                        LimasElips limas = (LimasElips) elips6;
                        return new Object[]{i + 1, f(a), f(b), f(tinggi),
f(limas.volumeLimasElips), f(limas.luasPermukaanLimasElips)};
                    }, errors);

            Thread t7 = buatThread("Limas Elips Terpancung", 6,
pbLimasTerpancung, hasilTerpancung, jumlahData, dataA, dataB, dataTinggi,
dataSudut, dataSemiMayorDalam, dataSemiMinorDalam, dataRadiusDalam,
dataFaktorAtas, isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips7 = new LimasElipsTerpancung(a, b,
tinggi, faktorAtas, manual);
                        elips7 = jalankanPolymorphism(elips7);
                        LimasElipsTerpancung lt = (LimasElipsTerpancung)
elips7;
                        return new Object[]{i + 1, f(a), f(b), f(tinggi),
f(faktorAtas), f(lt.volumeTerpancung), f(lt.luasPermukaanTerpancung)};
                    }, errors);

            Thread t8 = buatThread("Prisma Elips", 7, pbPrismaElips,
hasilPrisma, jumlahData, dataA, dataB, dataTinggi, dataSudut,
dataSemiMayorDalam, dataSemiMinorDalam, dataRadiusDalam, dataFaktorAtas,
isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips8 = new PrismaElips(a, b, tinggi,
manual);
                        elips8 = jalankanPolymorphism(elips8);
                        PrismaElips prisma = (PrismaElips) elips8;
```

```
                        return new Object[]{i + 1, f(a), f(b), f(tinggi),
f(prisma.volumePrismaElips), f(prisma.luasPermukaanPrismaElips)};
                    }, errors);

            Thread t9 = buatThread("Juring Elips 3D", 8, pbJuring3D,
hasilJuring3D, jumlahData, dataA, dataB, dataTinggi, dataSudut,
dataSemiMayorDalam, dataSemiMinorDalam, dataRadiusDalam, dataFaktorAtas,
isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips9 = new JuringElips3D(a, b, tinggi,
sudut, manual);
                        elips9 = jalankanPolymorphism(elips9);
                        JuringElips3D j3 = (JuringElips3D) elips9;
                        return new Object[]{i + 1, f(a), f(b), f(tinggi),
f(sudut), f(j3.volumeJuringElips3D), f(j3.luasPermukaanJuringElips3D)};
                    }, errors);

            Thread t10 = buatThread("Cincin Elips 3D", 9, pbCincin3D,
hasilCincin3D, jumlahData, dataA, dataB, dataTinggi, dataSudut,
dataSemiMayorDalam, dataSemiMinorDalam, dataRadiusDalam, dataFaktorAtas,
isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips10 = new CincinElips3D(a, b, tinggi,
radiusDalam, manual);
                        elips10 = jalankanPolymorphism(elips10);
                        CincinElips3D c3 = (CincinElips3D) elips10;
                        return new Object[]{i + 1, f(a), f(b), f(tinggi),
f(radiusDalam), f(c3.volumeCincinElips3D),
f(c3.luasPermukaanCincinElips3D)};
                    }, errors);

            Thread t11 = buatThread("Tembereng Elips 3D", 10,
pbTembereng3D, hasilTembereng3D, jumlahData, dataA, dataB, dataTinggi,
dataSudut, dataSemiMayorDalam, dataSemiMinorDalam, dataRadiusDalam,
dataFaktorAtas, isManual,
                    (i, a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, manual) -> {
                        Elips elips11 = new TemberengElips3D(a, b, tinggi,
sudut, manual);
                        elips11 = jalankanPolymorphism(elips11);
                        TemberengElips3D te3 = (TemberengElips3D) elips11;
                        return new Object[]{i + 1, f(a), f(b), f(tinggi),
f(sudut), f(te3.volumeTemberengElips3D),
f(te3.luasPermukaanTemberengElips3D)};
                    }, errors);

            Thread[] semuaThread = {t1, t2, t3, t4, t5, t6, t7, t8, t9,
t10, t11};
            for (Thread thread : semuaThread) {
                thread.start();
```

```
            }

            try {
                for (Thread thread : semuaThread) {
                    thread.join();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                SwingUtilities.invokeLater(() -> {
                    JOptionPane.showMessageDialog(GeometriGUI.this,
                            "Proses komputasi dihentikan paksa.",
                            "Interupsi",
                            JOptionPane.WARNING_MESSAGE);
                    btnGenerate.setEnabled(true);
                });
                return;
            }

            SwingUtilities.invokeLater(() -> {
                tambahBaris(modelElips, hasilElips);
                tambahBaris(modelJuring2D, hasilJuring2D);
                tambahBaris(modelCincin2D, hasilCincin2D);
                tambahBaris(modelTembereng2D, hasilTembereng2D);
                tambahBaris(modelBolaElips, hasilBola);
                tambahBaris(modelLimasElips, hasilLimas);
                tambahBaris(modelLimasTerpancung, hasilTerpancung);
                tambahBaris(modelPrismaElips, hasilPrisma);
                tambahBaris(modelJuring3D, hasilJuring3D);
                tambahBaris(modelCincin3D, hasilCincin3D);
                tambahBaris(modelTembereng3D, hasilTembereng3D);

                StringBuilder pesanError = new StringBuilder();
                for (int i = 0; i < errors.length; i++) {
                    if (errors[i] != null) {
                        pesanError.append("• Thread ").append(i +
1).append(": ").append(errors[i].getMessage()).append("\n");
                    }
                }

                btnGenerate.setEnabled(true);
                if (pesanError.length() > 0) {
                    JOptionPane.showMessageDialog(GeometriGUI.this,
                            "Proses selesai dengan error:\n\n" +
pesanError,
                            "Error Komputasi Thread",
                            JOptionPane.ERROR_MESSAGE);
                } else {
                    JOptionPane.showMessageDialog(GeometriGUI.this,
                            "Proses selesai! Semua data berhasil
ditampilkan.",
                            "Sukses",
                            JOptionPane.INFORMATION_MESSAGE);
```

```
                }
            });
        });

        masterThread.start();
    }

    private void resetTabel() {
        modelElips.setRowCount(0);
        modelJuring2D.setRowCount(0);
        modelCincin2D.setRowCount(0);
        modelTembereng2D.setRowCount(0);
        modelBolaElips.setRowCount(0);
        modelLimasElips.setRowCount(0);
        modelLimasTerpancung.setRowCount(0);
        modelPrismaElips.setRowCount(0);
        modelJuring3D.setRowCount(0);
        modelCincin3D.setRowCount(0);
        modelTembereng3D.setRowCount(0);
    }

    private void setMaksimumProgress(int max) {
        JProgressBar[] pbs = {pbElips, pbJuring2D, pbCincin2D,
pbTembereng2D, pbBolaElips, pbLimasElips, pbLimasTerpancung, pbPrismaElips,
pbJuring3D, pbCincin3D, pbTembereng3D};
        for (JProgressBar pb : pbs) {
            pb.setMaximum(max);
            pb.setValue(0);
        }
    }

    private void tambahBaris(DefaultTableModel model, Object[][] data) {
        for (Object[] row : data) {
            model.addRow(row);
        }
    }

    private Elips jalankanPolymorphism(Elips bentuk) {
        // Runtime polymorphism:
        // parameter bertipe Elips, tetapi method run() yang berjalan
mengikuti object aslinya.
        // Contoh: jika object aslinya BolaElips, maka yang dipanggil
adalah BolaElips.run().
        bentuk.run();
        return bentuk;
    }

    private interface HitungBaris {
        Object[] hitung(int index, double a, double b, double tinggi,
double sudut, double semiMayorDalam,
                double semiMinorDalam, double radiusDalam, double
faktorAtas, boolean isManual);
```

```
    }

    private Thread buatThread(String namaThread, int indexError,
JProgressBar progressBar, Object[][] hasil, int jumlahData,
            double[] dataA, double[] dataB, double[] dataTinggi, double[]
dataSudut, double[] dataSemiMayorDalam,
            double[] dataSemiMinorDalam, double[] dataRadiusDalam, double[]
dataFaktorAtas,
            boolean isManual, HitungBaris hitungBaris, Throwable[] errors)
{

        return new Thread(() -> {
            Random randDelay = new Random();
            try {
                for (int i = 0; i < jumlahData; i++) {
                    double a = dataA[i];
                    double b = dataB[i];
                    double tinggi = dataTinggi[i];
                    double sudut = dataSudut[i];
                    double semiMayorDalam = dataSemiMayorDalam[i];
                    double semiMinorDalam = dataSemiMinorDalam[i];
                    double radiusDalam = dataRadiusDalam[i];
                    double faktorAtas = dataFaktorAtas[i];

                    cekValidAngka(a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas, i + 1, namaThread);
                    hasil[i] = hitungBaris.hitung(i, a, b, tinggi, sudut,
semiMayorDalam, semiMinorDalam, radiusDalam, faktorAtas, isManual);

                    final int progress = i + 1;
                    if (i % 50 == 0 || i == jumlahData - 1) {
                        SwingUtilities.invokeLater(() ->
progressBar.setValue(progress));
                    }

                    if (randDelay.nextInt(100) < 3) {
                        try {
                            Thread.sleep(randDelay.nextInt(4) + 1);
                        } catch (InterruptedException e) {
                            Thread.currentThread().interrupt();
                            throw e;
                        }
                    }
                }
            } catch (Exception e) {
                errors[indexError] = new ThreadComputationException("Error
di " + namaThread + ": " + e.getMessage(), e);
            }
        });
    }

    private void cekValidAngka(double a, double b, double tinggi, double
```

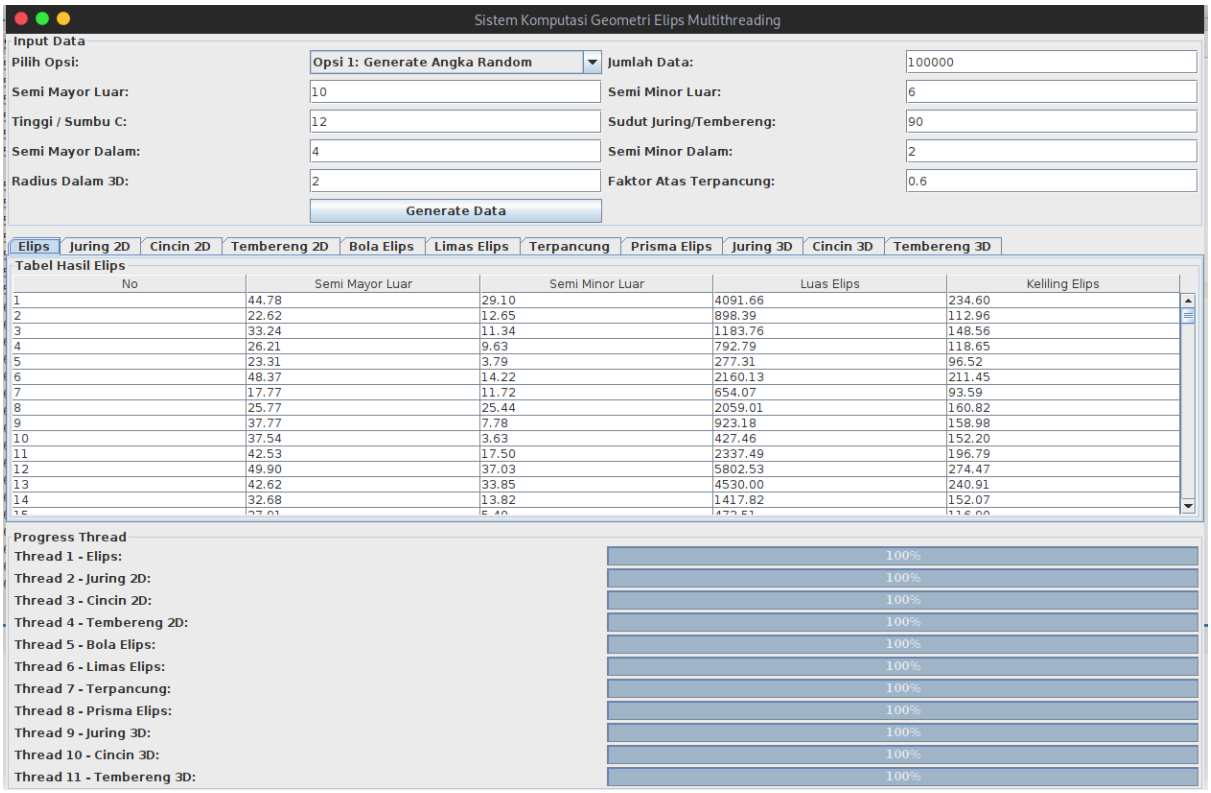
```
sudut, double semiMayorDalam, double semiMinorDalam,
            double radiusDalam, double faktorAtas, int noData, String
namaThread) {
        double[] values = {a, b, tinggi, sudut, semiMayorDalam,
semiMinorDalam, radiusDalam, faktorAtas};
        for (double value : values) {
            if (Double.isNaN(value) || Double.isInfinite(value)) {
                throw new ArithmeticException("Nilai NaN/Infinity
terdeteksi pada data ke-" + noData + " di " + namaThread);
            }
        }
    }

    public String f(double value) {
        return String.format("%.2f", value);
    }

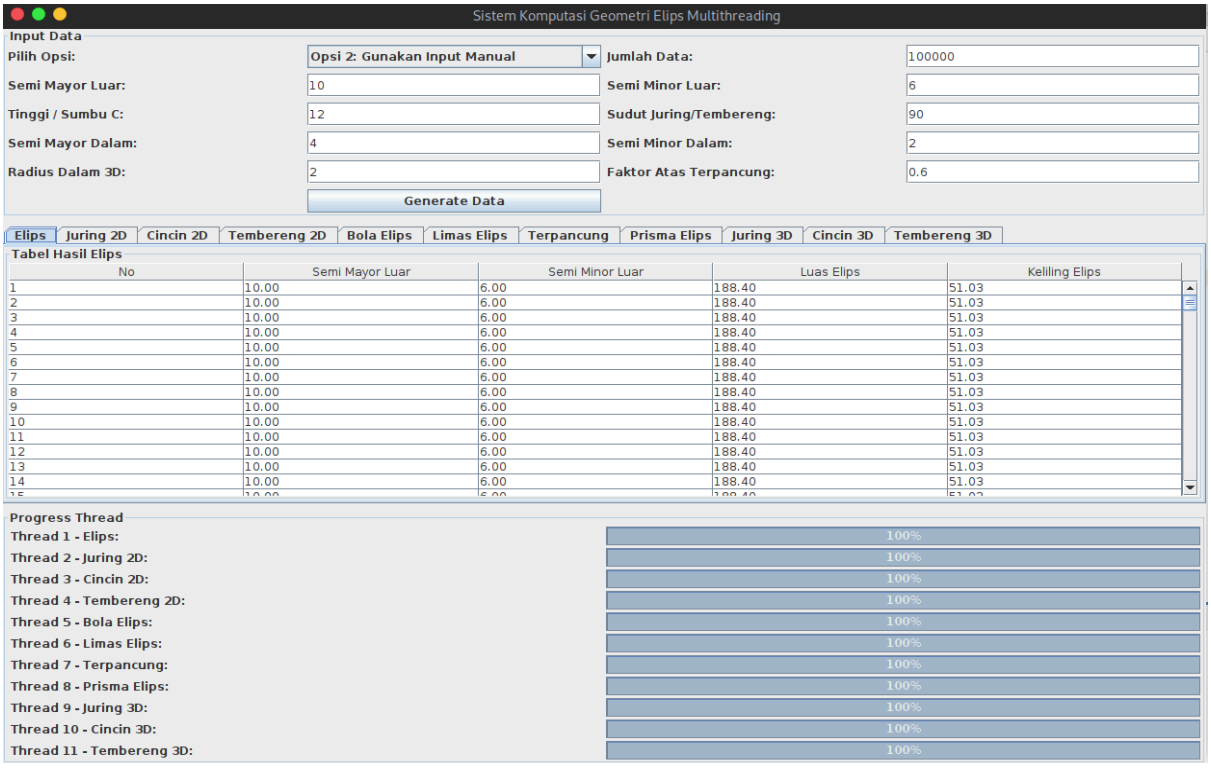
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new
GeometriGUI().setVisible(true));
    }
}
```

BAB IV
OUTPUT PROGRAM

4.1 Visualisasi Output (GUI)



Gambar 4.1 *Output Generate Angka Random*



Gambar 4.2 *Output* Input Manual